Evidence Before Impact: Upstream Change Intelligence for Dependency Decisions

Mehmet KIŞ
Ikarus AI Labs, Cambridge, United Kingdom*
mehmet@ikaruslabs.net https://ikaruslabs.net/

Technical Report TR-2026-01 30 October 2026 audit snapshot 8a9c3a8, current a1ecc58

Abstract

Dependency decisions rely on vulnerability advisories and lifecycle catalogues, yet several upstream changes that alter a dependency's trust, maintenance or availability appear in neither: a repository relocated or archived, or versioned images withdrawn from a namespace still in use. AI coding agents now make such decisions too. The central risk is a *false clear*: reporting a dependency as not affected when it is. We present OpenPulse, an open-source instrument that collects, qualifies and applies upstream change evidence to declared inventories. Completeness-aware negatives, conservative abstention and a third truth value are established ideas. Our contribution is a specific, testable model that combines them for dependency applicability: typed scope elements over heterogeneous identities, completeness with a recorded basis, and a negative only when a complete scope has every element decisively excluded, with a short conditional soundness lemma. An audit against this model found fifteen defects, twelve of which could produce or display false clears, some encoded as correct by the regression suite; all are fixed. An oracle written from the specification agrees with the implementation on 100,000 random cases and a 6,195-case matrix, where the 72 remaining differences are one documented, non-blocking label, and catches all 22 historical-defect and rule-disabling mutants. An exploratory retrospective study on 27 historical changes and 167 real dependency references found no false clear by the event-scope engine, where a naive reading of the same events produced 35 in 10 events. It also exposed a silent-miss defect, a discovery gap and a false clear in the separate advisory path, each since addressed. Its labels are the developer's, so it is not an effectiveness result; we specify a prospective evaluation, drafted for registration, that would provide one.

1 Introduction

Modern applications consist largely of third-party code, and each dependency carries assumptions about its upstream: who controls it, where it is published, whether it is maintained, and how long its release line is supported. Dependency decisions check those assumptions mostly against curated records. Software composition analysis (SCA) matches inventories against vulnerability databases such as NVD and OSV, and lifecycle tools consult catalogues such as endoflife.date. Several consequential changes appear in neither. Three examples from our own observations:

- **Namespace relocation.** The repository at `github.com/coreos/etcd` now resolves to `github.com/etcd-io/etcd`. References that record the old path keep working through a platform redirect, but they no longer name the canonical location, and their trust assumptions now depend on the redirect persisting.

*Ikarus AI Labs is a non-profit research initiative based in Cambridge, UK, working on evidence-based assurance for AI-mediated software change. We are open to collaboration on the evaluation and integrations described in this report: https://ikaruslabs.net/.

- **Archival.** A repository marked archived receives no fixes *in that repository* while it stays archived.
- **Distribution withdrawal.** In 2025 Bitnami announced that versioned images would move from `docker.io/bitnami` to an unsupported `docker.io/bitnamilegacy` namespace [3]. In October 2026 the former mainline repository `docker.io/bitnami/spark` listed no tags.

Detecting such a change is not enough. A dependency decision needs to know whether *this* dependency is within the change's scope, how well that is established, and what remains unknown. That step, *applicability*, has a security-relevant failure mode: the *false clear*, in which a dependency is reported as not affected when it is in scope. A false clear looks exactly like a correct negative. Name-based matching conflates `docker.io/redis` with `docker.io/bitnami/redis`, and an absent match is often read as an absent problem.

The question matters more as AI coding agents propose dependencies. Language models hallucinate package names that attackers can register [53]. Merely checking that a package exists does not defend against that, because an attacker-registered name passes an existence check. More generally, delegating dependency choices to agents creates an additional automated decision surface. How fast that surface grows, and what incident burden it carries, are empirical questions we do not answer. Our position is narrower: dependency existence and model confidence are insufficient, and current, scoped ecosystem evidence is an additional input to the decision.

Thesis and questions. OpenPulse investigates how current upstream change evidence can inform dependency decisions made by developers and AI coding agents, while preventing incomplete evidence from becoming a false clearance. The report is organised around three questions:

Q1. What upstream change evidence is missing from a dependency decision, and at which decision points? (Section 3)

Q2. How can that evidence be collected, qualified, and applied to a dependency without unsound negatives? (Sections 4–7)

Q3. What measurable difference does it make? (Sections 9–11)

Contributions.

1. A **typed applicability specification**. Membership is defined per scope-element kind (repository, tag, release line, content object, package, registry and project). Scope completeness has three states. An aggregation algorithm issues a negative only for a complete scope whose every element is decisively excluded. Two short lemmas prove that its negatives are sound given coverage and test soundness, and that its positives are sound given the separate premise of exactness. A table maps the premises to the components that must establish them (Section 5). Completeness-aware reasoning, conservative abstention and a third truth value are established ideas, and we do not claim them. What we add is their combination into a specific, testable model for dependency applicability: typed scope elements over heterogeneous identities, completeness with a recorded basis, exact elements kept apart from envelopes, and identity, freshness and consistency in one decision rule. Section 2.1 separates prior work from this addition and states, part by part, what the evidence does and does not yet show. The contribution is an assurance methodology: a property, an encoding that makes it machine-checkable, and an audit discipline. Of the report's parts, this applicability-decision policy and its audit are the primary contribution. Discovery, the evidence contract and the agent interface are supporting components and are not claimed as validated results. The lemma is a compact correctness argument for the aggregation rule, not the primary evidence of

OpenPulse's security. Explicit unknowns, provenance and hash-chained logs are established techniques that we integrate.

2. An **implementation audit** of OpenPulse against the specification. It found fifteen enumerated defects. Twelve could produce or display false clears, including expectations that the regression suite had encoded as correct; the other three are a false impact (D7) and two under-positives (D14, D15). All fifteen have regression fixes. A specification-conformance oracle, run over a generated matrix, found three of them and four inconsistencies in the specification itself. The negative-admission conditions are now enforced, with stated gaps: the freshness bound is opt-in in the engine (the agent interface applies one by default since `323bb3c`), contradictions are detected only within an event's own evidence, and declarations are recorded but not verified. The audit also found an over-broad matching rule (D7), which is fixed: element roles now keep envelope membership from producing positives (Section 6).

3. A **decision-point model** for consuming the evidence before adoption, before installation and after adoption. It covers the AI-specific risks the evidence does and does not address, and distinguishes the structured fields of the agent interface from untrusted upstream text (Sections 3 and 7).

4. A **development case ledger** and a **draft prospective evaluation protocol**, including a specification-conformance suite and an agent decision experiment, to measure whether the evidence changes decisions relative to a simpler conservative baseline (Sections 9 and 10).

5. An **exploratory retrospective study** on 27 historical upstream changes from archived primary sources and 167 real dependency references, frozen before any policy ran (Section 11). It reports the safety and coverage of OpenPulse against three baselines, every representation gap, a discovery measurement, and the five defects it and its follow-up exposed, one of them a false clear in the advisory path. Its labels are the developer's, so it is exploratory, not confirmatory.

Status. Table 2 separates what is specified, what is implemented at the pinned commit `8a9c3a8`, and what is open. The report's security claim is conditional. It establishes that a negative verdict is sound *if* the premises of Section 5.4 hold, and audits which of them the implementation establishes. It does not show that the deployed system meets them often enough to justify its negatives; that is an empirical question for Section 10. Limitations are collected in Section 12 and are not repeated throughout. The report uses two snapshots of `https://github.com/IkarusAILabs/OpenPulse` (Appendix C). The *audit snapshot* `8a9c3a8` is the commit every statement "at `8a9c3a8`" refers to, and the commit at which the fifteen enumerated defects were found and fixed. The *current snapshot* `a1ecc58` adds the changes in Table 1, made after the audit. The conformance, randomised, re-injection, version and cost measurements were repeated at `a1ecc58`, and the counts reported for them are from that run; where they differ from the audit snapshot, both are given. The retrospective study of Section 11 is pinned to its own freeze tag `retro-freeze-2026-10-30` (`3639a0f`), between the two. The latest tagged release, v0.7.1, predates the fixes. OpenPulse is released under the Apache License 2.0.

Terminology. A *false clear* is a NotAffected verdict for a dependency in the event's scope; a *missed detection* is an upstream change that discovery never reports. Event confidence grades the evidence for an event, and verdict confidence grades one dependency's verdict. *Complete* has three senses that do not imply one another: acquisition coverage, scope completeness and inspection coverage. An event's *asserted* scope is what its sources say; its *established* scope is the part whose completeness has a recorded basis; and its *evaluable* scope is the part OpenPulse can test with the semantics it implements. A negative needs the whole asserted scope to be

Change	What it does	Error it removes
F1: identity per ecosystem	A package resolves to a project only through a curated (ecosystem, name) pair; every catalogue entry now lists its packages. PyPI `redis` (redis-py) had been treated as the Redis server	False impact
F2: project package element	A package-scope value that names the event's own project denotes every package curated as that project; previously applied but unspecified	Specification gap
F3: qualified package values	A purl value (`pkg:pypi/redis`) decides by ecosystem and name before identity	Name-only matching
F4: self-resolved packages	A package that resolves only to itself is RELATED even when named like the project (the package-side twin of D10)	Name taken as identity
Contract 1.2.0	Adds `verdict_meaning` (direct scope; embedded components, availability, maintenance and security consequence marked not evaluated), `scope.completeness` with its basis, and the evaluation context	Bare labels read as permission
D-R1, D-R2	A Go module reaches the source repository it lives in (positively only); the GitHub collector follows one same-origin redirect, so transfers are observed (Section 11)	Silent miss; missed relocation
D-S1–D-S3	OSV advisories without a CVE are kept; another package's range in the same advisory is ignored; version-only entries follow the OSV schema (Section 11)	**False clear**; false impact
Scoped npm identity	`@scope/name` keeps its scope instead of resolving to the empty string	Shared empty identity
Discovery	Targets come from the inventory (Go modules, repository references, Docker Hub images), and every npm, PyPI, Maven, Go and crates.io package is queried on OSV by name	Uncatalogued projects unseen
Catalogue-map cache (`0174a26`, after `a1ecc58`)	The package, curated-slug and alias maps are memoised per catalogue object instead of rebuilt on every identity resolution	1.8× matching slowdown from F1
Agent freshness bound (`323bb3c`, after `a1ecc58`)	The agent tools run in a *current* mode with a 90-day evidence-age bound by default, or an explicit *replay* mode without one that says so; responses carry the evaluation context and each cause's scope completeness	Agent negatives on evidence of any age
Evaluation	Strong baseline (v) with pinned comparators, an independent oracle kit, a registration draft and the retrospective study	–

Table 1: Changes from the audit snapshot `8a9c3a8` to the current snapshot `a1ecc58`. Each has regression tests; the oracle, the conformance matrix, the randomised generator, defect re-injection and the strong baseline were extended for F2–F4.

established and evaluable. Unsupported or unevaluable elements are kept and give `?`, which blocks a clearance. The envelope is never narrowed to the subset the implementation can evaluate. *Artefact* means a distributable object, and `artifact` is its spelling in code identifiers.

Component	Specified	Implemented at `8a9c3a8`	Open
Scope typing	Repository, namespace, tag, release-line, object, package, registry, project elements, each *exact* or *envelope*	Artefact elements with roles by form; tag predicates `<version>` and `*`	Roles not labelled in the contract
Canonical form	Per element kind, shared with the oracle	Lowercase; scheme, tag, digest stripped; `docker.io` default; Docker Hub host aliases folded	Package-name forms per ecosystem (addressed at `a1ecc58`: F1–F3)
Completeness	Verified / declared / partial	Opt-in declaration; unspecified is partial	Declarations are not verified
Aggregation	Every element evaluated; negative needs complete scope	Yes; regression- and conformance-tested	–
Cross-event summary	No negative masks an unresolved association	Summary step, checked exhaustively; unmapped identity is RELATED	–
Versions, OSV ranges	Union of intervals; unsupported forms undetermined	Yes; each range compared under its own scheme (SemVer, PEP 440, plain numeric)	Differential against `packaging` (PEP 440), and in CI against the pinned npm, Go and Maven tools; others untested
OCI digests	Directional, platform-aware containment	Interim: unequal digests undetermined	Level resolution; no digest-based negatives meanwhile
Freshness	Negatives only from current, non-superseded events	Withdrawn and superseded events dropped; opt-in age bound on negatives	Bound off by default; one bound, not per class
Contract	Evidence, unknowns, negative semantics	v1.1, unsigned (v1.2.0 at `a1ecc58` adds verdict meaning and completeness)	Supersession, revocation; negative-semantics fields at `8a9c3a8` (in 1.2.0)
First detection	Externally witnessed	Unkeyed local ledger	Backdating not detectable

Table 2: Specification versus implementation at `8a9c3a8`. *Implemented*: present at `8a9c3a8` and covered by regression tests. *Interim*: a fail-closed substitute that withholds negatives where the specified semantics are missing (Table 16). *Open*: specified but not implemented, or not established.

2 Background and Related Work

Abandonment and deprecation. Empirical work studies how projects go dormant [57], how they survive the loss of core developers [2], how practitioners perceive the bus factor [25], and how npm packages are deprecated [7]. Developers lack systematic ways to learn that a dependency was abandoned [35]. These studies characterise the process. We detect observable end states and relate them to inventories.

Dependency management. Dependency networks differ in size, growth and update practice across packaging ecosystems [13], and developers often keep outdated dependencies, frequently because they are unaware that an update or a migration is needed [28]. That unawareness is the gap this report addresses for changes that are not vulnerabilities: the question is not whether to update, but whether a given upstream change applies to a given declared dependency at all.

Supply-chain attacks and namespaces. Attack taxonomies [40, 29] and ecosystem studies [62, 60, 22] identify account takeover, abandoned projects and namespace reuse as vectors.

GitHub documents that transfers leave redirects, that creating a repository at the old location removes them, and that some names are retired against reuse [18]. Industry research describes the resulting "repojacking" risk [19].

Applicability and vulnerability data. Vulnerability applicability has been studied at the level of code: impact assessment and reachability analysis decide whether vulnerable code is present and used [45, 46, 47]. That is a finer question than ours, and a complementary one. It assumes that the vulnerable component has already been associated with the dependency, which is the step at which our false clears arise. Naive counting inflates vulnerability figures [43], and SCA tools disagree [24]. OSV [41], GHSA [17], NVD [37], KEV [10] and endoflife.date [15] are the dominant feeds. NVD data has known inconsistencies [1], and prior work measures lags between events and records [33, 4].

Provenance, posture and logs. in-toto [56], SLSA [52] and Sigstore [38] establish build provenance. CycloneDX [14] and SPDX [55] standardise inventories. Empirical studies of SBOM practice report that producers and consumers struggle with the completeness and accuracy of the inventories themselves [58, 54]. Our setting has the same problem on the other side of the match: an inventory may be incomplete, and so may an event's affected set. Only the second is modelled here; inventory coverage is reported as a separate qualifier (Section 7). Scorecard [59] and deps.dev [20] provide posture signals. Hash-chained logs [50], history trees [9] and Certificate Transparency [30, 31] are the standard tamper-evident designs.

AI agents and containers. AI-assisted code is often insecure [44]. Models hallucinate package names [53, 27], and agents that read external content are exposed to indirect prompt injection [21, 12]. This work studies whether a recommended package exists or is malicious. It does not ask whether a real, existing dependency is affected by an upstream change after the model's knowledge was fixed, which is the input our model supplies to such a decision. Container studies examine image content [51, 61, 5, 34] rather than changes to how pinned images are distributed.

Completeness reasoning. The underlying principle is old. Under the closed-world assumption, absence from a database is read as falsity [49]. Without it, a negative conclusion needs explicit knowledge of completeness. A third truth value for "unknown" goes back to Kleene's strong three-valued logic [26] and to null values in the relational model [6], and incomplete-information databases make the distinction formal [23]. Completeness assertions attached to parts of a database say where absence may be read as falsity [36]. Local completeness statements let a query answer be declared complete from incomplete sources [32], and later work decides when a query over partially complete data is itself complete [48]. Local closed-world reasoning lets a planner draw negative conclusions only where its information is known to be complete [16]. Our coverage premise is an over-approximation in the sense of abstract interpretation [8]: sound for exclusion, and silent about membership. We claim no new logical principle. Our contribution is to apply a local completeness assertion to the affected set of an upstream change: a typed representation over heterogeneous dependency identities, an implementation discipline and an audit against it. Whether it improves decisions is the empirical question of Section 10.

Closest semantics. VEX [11, 42] is the closest representation of applicability. It has an explicit unresolved status (*under investigation*). The CISA minimum requirements attach a justification or impact statement to *not affected*, which preserves a rationale for a negative. VEX documents are versioned, and we have not verified how consistently consumers invalidate earlier statements. CSAF [39] carries the same statuses (known affected, known not affected, fixed, under investigation) over a product tree whose branches can name vendors, products and

Representation and consumer policy	`bitnami/redis:7.2`	`bitnami/nginx`	`docker.io/redis`
(1) Affected list of five images; absence read as not affected	affected	not affected (false clear)	not affected
(2) Same list as VEX; unlisted treated as under investigation	affected	under investigation	under investigation
(3) Namespace elements, χ declared (announced basis)	affected	affected	not affected

Table 3: One event, three representations. Ground truth: both Bitnami images are in scope, and `docker.io/redis` is not.

version ranges; status is still asserted per product in that tree. Two properties we did not find in the formats we reviewed (OSV, VEX and OpenVEX, CSAF, CycloneDX): a representation of whether an *affected-object set is complete*, so that a negative can be conditioned on it, and one applicability rule relating an upstream change to *heterogeneous* dependency identities. This is a statement about those formats, not about every tool. Table 4 compares *stated design scope*. It is not a benchmark.

The contribution is a *property* and an *encoding* of it, not a new data format. The property is that a negative verdict is conditional on a stated completeness of the affected set. The encoding is the pair (E, χ) with typed elements. Existing formats can approximate it. A VEX producer can list every product with an explicit status. A consumer policy can treat every unlisted product as *under investigation*. A justification field can carry a free-text claim of completeness. What none of these provides is a machine-readable statement, attached to the affected set, that tells the consumer whether absence from the list may be read as exclusion.

Table 3 works one case through three representations: Bitnami's catalogue-wide change, with three dependencies. (1) A conventional affected-product list of five images, read with absence as not affected, clears `docker.io/bitnami/nginx`. That is a false clear, and it is defect D4 in our own implementation (Section 6). (2) The same list, as VEX statements consumed under a conservative policy that treats unlisted products as under investigation, is safe. But it cannot clear `docker.io/redis`, which is plainly outside the change, unless the producer enumerates every unaffected image in existence. (3) With typed elements and χ, the namespace element declared complete makes `bitnami/nginx` affected and clears `docker.io/redis`. The same five-element list with χ partial behaves like (2). The difference between (2) and (3) is decisiveness at equal safety, not safety. That is exactly what the comparison with the strong conservative baseline in Section 10 measures. What (3) makes machine-checkable is that the license to read absence as exclusion travels with the affected set, as data with a recorded basis, instead of living in each consumer's policy. A conservative consumer policy remains expressible in existing formats. We have not found a way to state "every image under this namespace" with the product identifiers of the formats we reviewed. Where that is possible, (2) can approach (3).

2.1 What is new, and what it enables

Completeness-aware reasoning, conservative abstention and a third truth value for "unknown" are established ideas (above), and we claim none of them. Our claim is narrower and testable: a specific *model for dependency applicability decisions*, made precise enough that an implementation can be checked against it and that it can be compared with simpler policies on real changes. Table 5 separates, for each part of that model, what is prior, what this report adds, how the addition is tested, and what the evidence shows so far. Each row is a candidate contribution, and several are not yet validated.

Approach family	Disc.	Ident.	Appl.	Unkn.	Temp.	Export
Vulnerability feeds (OSV, GHSA, NVD)	–	P	P	–	P	Y
Lifecycle catalogues (endoflife.date)	–	P	P	–	Y	P
SCA tools	–	P	Y	P	–	P
Reachability analysis [46]	–	P	Y	P	–	–
VEX [11, 42]	–	P	Y	Y	P	Y
Posture (Scorecard, deps.dev)	P	P	–	–	–	P
Provenance (in-toto, SLSA, Sigstore)	–	Y	–	–	P	Y
OpenPulse (this report)	P	Y	Y*	Y	P	P

Table 4: Stated design scope, taken from each family's documentation; not a measured comparison. Disc.: discovery of non-lifecycle upstream change. Ident.: identity across namespaces and pinning forms. Appl.: event-specific applicability. Unkn.: explicit unknown. Temp.: provenance of dates. Export: verifiable machine-readable output. Y: primary goal; P: partial; –: outside stated scope (absence not tested). *Implementation gaps in Table 2. The families are of different kinds and are not interchangeable: OSV, VEX, CycloneDX and SPDX are data formats; NVD, GHSA, KEV and endoflife.date are curated feeds; SCA tools and reachability analysis are consumers that apply a policy; Scorecard and deps.dev are posture signals; and in-toto, SLSA and Sigstore are provenance and build assurance. The text after this table gives a worked example of the behavioural difference.

What the model lets one do. The question a reader should ask is what a researcher or practitioner can do with this model that existing approaches do not do as reliably. Our answer has five parts, each with its evidence and its limit.

1. *Issue a negative whose justification travels with the data.* The completeness state and its basis are part of the affected set, so any consumer can mechanically reject a NotAffected whose scope is not complete on a recorded basis. In existing formats that rule lives in each consumer's policy (Table 3). On 27 historical events, reading affected lists naively produced 35 false clears in 10 events; the model produced none. A conservative policy that never clears is equally safe, so this part is about making negatives auditable, not about safety over every alternative.

2. *Clear what is plainly out of scope without enumerating it.* Namespace, registry and release-line elements with a declared complete scope let a consumer clear `docker.io/redis` under a Bitnami change at equal safety (Table 3). Against a policy that never clears, the retrospective gain in resolution was 0.091 at the freeze (below the pre-specified 0.10) and 0.170 after two defect fixes, a post-hoc figure.

3. *Keep identity uncertainty visible instead of turning it into exclusion.* The identity rule is the part with the clearest evidence: ablating it, so that an unresolved package or image counts as unrelated, silently dropped 18 more in-scope pairs in the retrospective study.

4. *Test an implementation against the decision rule.* Because the model fixes what absence means for each element kind, an oracle can be written from the text and compared with any implementation. Doing so found defects that the regression suite had encoded as correct (Section 6), and every re-injected mutant is caught. Formats that leave absence to the consumer give no such reference to test against.

5. *Say what cannot be stated.* A scope that the language cannot express exactly is stated as a safe over-approximation and recorded as a gap, never silently narrowed. The retrospective study found four such forms, which bounds the model's reach rather than hiding it.

What the model does not do is equally specific. It does not decide whether vulnerable code is reachable. It does not by itself make decisions better than a careful implementation of the same

Candidate contribution	Established idea	What this report adds	Evidence so far
Representation of scope and its completeness	Completeness assertions on parts of a database [36, 32]	Typed scope elements over heterogeneous dependency identities (repository, tag, release line, object, package, registry, project), with completeness as a state plus a recorded basis, exported in the evidence contract (Sections 5.2 and 7)	Conformance: 100,000 random cases agree with an oracle; of 6,195 matrix cases, 6,123 conform and 72 differ by one non-blocking label, none unsafely. Representability: 27 real events; four scope forms could not be stated exactly (Section 11)
Conditions for a justified negative	Closed-world and local closed-world reasoning [49, 16]	Admission conditions N1–N5 and a lemma separating the aggregation argument from the premises an implementation must establish (Section 5.4)	Proof is conditional. On 27 events, assuming completeness everywhere produced no additional false clear, so the condition's safety value is not yet isolated empirically
Exact elements versus envelopes	Over-approximation in abstract interpretation [8]	Roles by element form: an envelope may exclude but never yields a positive; an exact element may do both (Section 5.2)	The over-match D7 was found and fixed; the D7 and role mutants are caught. Retrospective false impact 1.6% against 3.2% for the list-based policies
Identity, freshness and consistency in one decision	Conservative handling of unknowns; VEX *under investigation* [11]	Unresolved identity never excludes (D10, F4); stale, withdrawn, superseded and contested events never clear (N4, N5); per-ecosystem package identity (F1–F3) (Sections 5.5 and 4.4)	Identity: treating unresolved identity as unrelated raised silent misses from 11.7% to 29.1% of in-scope pairs. The freshness ablation changed no decision, and no event was contested, so those rules were not exercised
Implementation, audit and evaluation	Differential and mutation testing; prospective evaluation	An oracle written from the specification, a defect re-injection suite, a registration draft, and an exploratory retrospective study (Sections 6, 10 and 11)	22 of 22 historical-defect and rule-disabling mutants caught. Retrospective: no false clear where a naive list reading had 35 (10 events). Pre-specified test not yet run; labels not independent

Table 5: What is prior, what this report adds, and what is validated. "Evidence so far" states results and their limits; none of the rows is established by the prospective, independently labelled evaluation that Section 10 specifies.

model: the strong baseline (v) uses the same scope language and differed from OpenPulse by 0.030 in resolution at the freeze. So the contribution is the model and its checkability, not a superior engine. And the study could not attribute the absence of false clears to the completeness condition alone, because typed elements and identity rules act together; isolating it needs events whose published affected lists are incomplete, which the prospective evaluation is designed to sample.

Risk	Relation to OpenPulse
Invented or attacker-registered package names	Motivation only. Not a demonstrated OpenPulse capability; existence checks alone are insufficient [53].
Existing dependencies with stale maintenance, namespace or distribution assumptions	Directly addressed by the implemented change classes (Section 9).
Prompt injection through dependency intelligence	Created by agent-facing consumption; acknowledged, untested (Section 7).

Table 6: Risks in AI-mediated dependency selection.

3 Q1: What Evidence Is Missing from a Dependency Decision

Change classes. An *upstream change* alters the state of a project or its distribution channels in a way that can change the risk, operability or legal standing of dependents. OpenPulse *discovers* three classes: namespace relocation and archival on GitHub, and distribution change on Docker Hub. At `8a9c3a8` it watched catalogued projects only; at `a1ecc58` it also derives what to watch from the inventory (Go module paths, `github.com` references and Docker Hub images). End-of-life, end-of-support and vulnerability events come from existing feeds, and at `a1ecc58` every npm, PyPI, Maven, Go and crates.io package is also queried on OSV by its own name. Licence and support-model changes can be represented and matched only as curated events.

Decision points. The evidence is relevant at three stages, and OpenPulse contributes evidence to each one without owning enforcement or choosing replacements:
1. *Before adding* a dependency: is the proposed identity what it appears to be, and what current upstream evidence exists about it?
2. *Before installing or executing* it: is there enough evidence, and enough approval, to proceed?
3. *After adoption*: which upstream changes now apply to the actual inventory?

Risks. Table 6 separates the risks that motivate the work from those the implemented evidence addresses.

Design constraints. (C1) No claim without named evidence, and weak evidence never reaches action-level channels. (C2) An undetermined question is never reported as a negative. (C3) Verdicts are deterministic functions of recorded inputs, with no language model in the decision path. (C4) Output is evidence and a recommendation to investigate. Nothing is remediated automatically.

4 Q2a: Collecting and Qualifying Evidence

4.1 Pipeline and trust boundaries

Collectors acquire data from GitHub, OSV, NVD, MITRE CVE, CISA KEV, endoflife.date and the Docker Hub/OCI registry. Pure-function analysts derive *findings*. A gate vetoes events whose claims exceed their support. The verdict engine relates gated events to inventories. Renderers produce reports, alerts, evidence contracts and agent responses (Figure 1). A failed acquisition becomes a recorded gap, never an empty result.

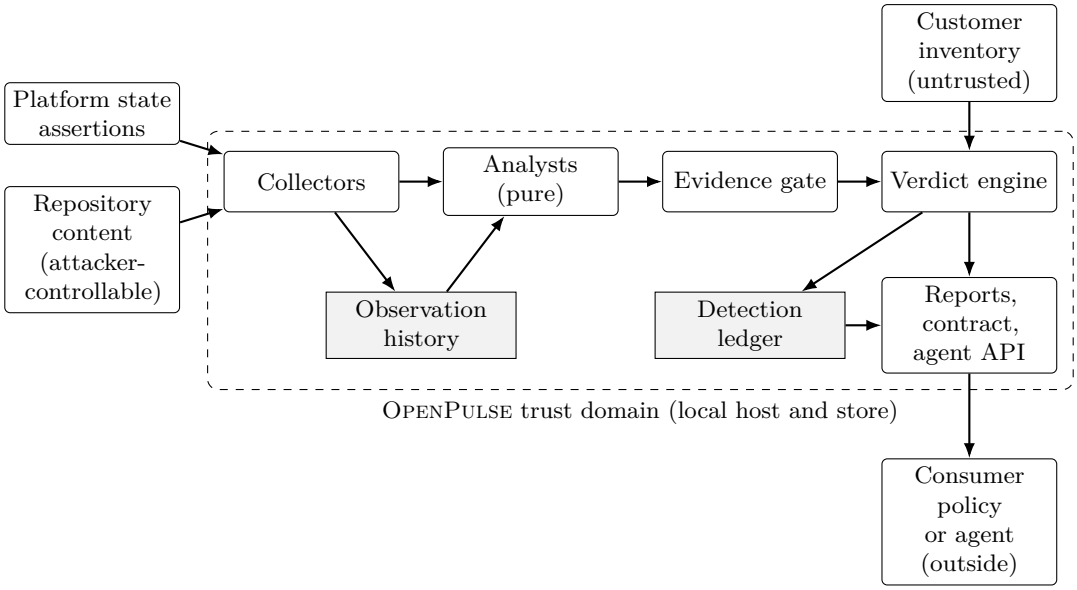


Figure 1: Pipeline and trust boundaries. Platforms are trusted to report their own *state* truthfully at query time: canonical path, archived flag, tag list. They are not trusted to be complete or current. Repository and advisory *content*, such as descriptions, release notes and names, is attacker-controllable and is treated as data, never as instructions. The stores protect only against accidental corruption. Policy is outside OpenPulse.

4.2 Observation, claim, event

An *observation* records what a source exposed at time t. A *claim* is an assertion that names its supporting evidence. An *event* is a gated, scoped and dated unit of intelligence. A diff between observations is only a candidate change, with an explicit evidence strength, until it is gated.

4.3 Authority, independence and confidence

Sources are tiered as official, primary, secondary or tertiary. Authority is claim-specific in principle: GitHub is authoritative for a repository's canonical path, not for why it moved. Corroboration counts independent source families, folding derivation chains, and NVD and OSV count as one family. Where no family is declared, the implementation falls back to the source name, so undeclared copies of one origin can count as independent. Confidence (Unverified < Emerging < Corroborated < Confirmed) and impact (Informational < Watch < Review < Action < Critical) are *policy categories*, not calibrated probabilities. Table 7 collects every rule. A verdict's confidence is

$$\mathrm{conf}(d, e) = \min\big(\mathrm{ceil}(\mathrm{level}(d, e)),\ \mathrm{conf}(e),\ \mathrm{cap}_{\mathrm{id}}(d, e)\big). \tag{1}$$

Here $L = (\text{Unverified} < \text{Emerging} < \text{Corroborated} < \text{Confirmed})$ is a total order, and min is its meet. $\mathrm{conf}(e) \in L$ is the gated event confidence. ceil maps the verdict's match level to L by the "match ceiling" row of Table 7. $\mathrm{cap}_{\mathrm{id}}(d, e)$ is Emerging when the verdict is affected through a Review-Required or Unverified identity mapping, other than by exact artefact equality, and Confirmed (no effect) otherwise. The equation is normative, and it is also what the implementation computes (`core/risk/check.py`). Impact is copied from the gated event and never raised by the verdict engine. Equation (1) involves no numerical thresholds. It takes the minimum over an ordinal scale, so each cap can only lower a verdict's confidence. Neither confidence nor impact has a numerical interpretation, in outputs or in evaluation tables. The ceilings and caps are design choices, not calibrated values. Whether they separate reliable verdicts from uncertain ones is measured by the stratified error rates in Section 10.

Stage	Rule
Event confidence	CONFIRMED: official evidence (official or primary for lifecycle classes). CORROBORATED: $\geq$2 independent families. EMERGING: one credible source. UNVERIFIED: otherwise.
Gate	UNVERIFIED $\Rightarrow$ impact $\leq$ WATCH; EMERGING $\Rightarrow \leq$ REVIEW; ACTION/CRITICAL need $\geq$ CORROBORATED; artefact classes must name artefacts; dated classes need dates; unresolved contradiction blocks ACTION/CRITICAL.
Vulnerabilities	Keyword-only match $\leq$ REVIEW; KEV with undetermined applicability $\Rightarrow$ REVIEW.
Match ceiling	object, repository, exact namespace or tag $\rightarrow$ CONFIRMED; version, not-affected $\rightarrow$ CORROBORATED; package, project, exact registry $\rightarrow$ EMERGING; related, unknown, and any envelope membership $\rightarrow$ UNVERIFIED. At `8a9c3a8` registry membership is reported as related.
Identity cap	Affected through a REVIEW-REQUIRED or UNVERIFIED mapping $\Rightarrow \leq$ EMERGING.

Table 7: All confidence and impact rules.

4.4 Temporal roles and lead time

Five roles are kept apart: *announced* (from sources with a trustworthy publication date, otherwise unknown), *effective*, *first detected*, *last verified* and *reported*. Conflicting effective dates are preserved as separate evidence. Detection lead is $\text{lead}(c) = \text{effective}(c) - \text{firstDetected}(c)$ in days. Positive means advance warning, negative means a measured late detection, and the lead is undefined only when a term is unknown. A lead depends on when monitoring started and how often it polls, so it is an *instrument capability and evaluation metric*, not an established benefit.

Freshness and supersession. A historical observation presented as current can mislead, for instance about where a repository now lives or whether a tag still resolves. We specify freshness per source class s with a policy bound τ_s. An observation is *fresh* while its last successful acquisition is within τ_s of the evaluation time. A failed acquisition records a gap and never refreshes it. An event is *current* if its *last verified* date is fresh and no later event from the same source corrects, withdraws or supersedes it. A negative verdict is admissible only from a current event. From a stale event the verdict is RELATED, with the reason *stale* kept distinct from *undetermined*, so that a consumer can tell old evidence from unresolved evidence. A withdrawn advisory asserts nothing. At `8a9c3a8` part of this exists. Verdicts are recomputed from the loaded events on every run, and nothing stores a negative for reuse, so a corrected event changes the next run's verdict. The contract exports `last_verified`. Withdrawn OSV entries are skipped, and withdrawn or superseded events are not evaluated. The bound is implemented as an opt-in freshness check. Given an evaluation time and a maximum evidence age, a negative from an event whose newest evidence is older becomes RELATED, with the reason *stale*. It is off by default, because replaying recorded evidence offline would otherwise make every negative stale, and it is one bound rather than one per class. The report-placement freshness classes in `core/freshness.py` decide whether a change is *news*. They are not evidence currency and do not gate verdicts. An exported contract has no expiry or supersession, and re-querying does not guarantee newly acquired evidence either. The agent interface reuses each fetched response, successful or failed, for up to an hour, and offline runs use whatever events are loaded (Section 7). Recomputation is not freshness. A verdict recomputed today from an observation acquired weeks ago is as old as that observation. Unless the caller sets the bound, a negative can therefore still rest on evidence of any age. Since `323bb3c` the agent interface sets one by default (Section 7); the engine, the command line and the contract export do not. If a source is unavailable for several days, its acquisitions are recorded as gaps, the events already loaded stay in force, and nothing marks the resulting negatives as stale. Without a bound, the assurance a negative provides is limited to evidence whose currency the consumer has established, for example by checking

`last_verified` against its own bound. Enforcement, contract expiry, explicit supersession and a stale failure state are prerequisites for using negatives as automated policy inputs.

Event classes need different currency rules. A relocation is a historical fact. Its observation stays true of the time it was made, and what can go stale is whether the redirect still holds. Availability of a tag or image is a current state, and needs a short bound. A scheduled lifecycle statement is valid until its source revises it, and needs re-verification of the schedule, not of the date. There is also a time-of-check to time-of-use gap across the decision points of Section 3. A fresh answer before adoption says nothing about the state at installation. A bounded implementation therefore binds each assessment to the exact reference, ideally a digest, and to its evaluation context and acquisition time. Stage 2 must re-evaluate it rather than reuse a stage-1 answer. After a failed acquisition, no cached negative is reused silently: the result is *stale* or *unavailable*, and both are distinct from undetermined.

4.5 Evidence integrity

Registry observations are hash-chained, $H_n = \mathrm{SHA256}(\mathrm{canon}(o_n) \| H_{n-1})$, and broken chains are never diffed against. A change to acquisition semantics bumps a parser version and forces a re-baseline. First detections are earliest-witness ledger entries whose content hash is verified on read. The hashes are unkeyed. They detect accidental and partial corruption, not a writer who rewrites the suffix, rolls back, or backdates a fact. Rollback and backdating resistance need witnesses outside the operator's control, such as a monotonic counter or the chain tip anchored in an external log [31]. A keyed MAC alone would not prevent replay of an older store. The guarantee is therefore *integrity against accidental and partial corruption only.* It establishes neither authenticity (who wrote the evidence), nor freshness, nor rollback resistance, and a privileged local operator, or a compromised host, is outside the adversary it resists. We call the stores hash-chained rather than tamper-evident for this reason.

5 Q2b: Applying Evidence — Applicability Specification

5.1 What a verdict means

Four predicates are kept apart: (a) *object equality*; (b) *scope membership*; (c) *availability*, meaning that a reference resolves under stated conditions; and (d) *consequence* for the dependent system. *Affected*(d, e) means (b). NotAffected(d, e) means only that d is outside e's *directly named* scope. It says nothing about availability, maintenance or vulnerability, and nothing about other events. Table 8 states what each verdict asserts and what is required before it may be issued. Every condition in its last column is checked in code and exercised by the build gate (Section 6).

Whether an event reaches a downstream distribution or an embedded component must come from the event's definition, not from identity modelling alone. An upstream end-of-life statement may concern the upstream's maintenance obligations, not a vendor's support of a repackaged image. A "contains" relation would let OpenPulse ask the question, but containment should not propagate every event automatically. Each relation needs event-specific rules. A negative verdict should therefore carry three distinct statements: outside the directly named scope; embedded-component relation not evaluated; and operational or security consequence not evaluated. Agents in particular may read a bare NotAffected label as permission to proceed. The v1.1 contract carries only the relationship name; contract 1.2.0, at `a1ecc58`, carries the three statements as `verdict_meaning`.

The same boundary applies to positives. AffectsArtifact means scope membership, not that a reference is broken, unsafe, vulnerable or unmaintained. Case 10 (Table 20) shows that a tag listing can establish a distribution change without establishing whether a particular reference still resolves. The contract should therefore carry four separate fields, each with its own evidence or an explicit *not evaluated*: direct scope membership; observed availability

Verdict	Asserts	Required before it may be issued
AFFECTS*	d is in the event's scope	Membership in an *exact* element, after canonicalisation; for versions, a comparison under the range's own scheme
AFFECTS-PROJECT	d belongs to the event's project; scope membership not established	Identity resolved to the project; no exact member, nothing undetermined
RELATED	An association exists but is not established	Any element undetermined: envelope membership, unresolved identity, incomparable version, mixed pinning, unknown tag, or a negative withheld (partial scope, missing basis, stale or contested event)
NOTAFFECTED	d is outside the directly named scope	N1–N5: every element excludes; identity exclusion only through a curated mapping; complete scope with recorded basis; fresh and uncontested event
UNKNOWN	No association	d is curated as another project, or no element relates to it; never because identity failed to resolve

Table 8: Verdict semantics. Across events, a dependency is affected if any event affects it, cleared only if every associated event gives NOTAFFECTED, and unresolved otherwise (Table 10).

or resolution; maintenance or support implication; and security consequence, only where it is independently established. That would stop a downstream policy from turning a correct membership result into an unsupported security conclusion. These fields are not in contract v1.1 at `8a9c3a8`. Contract 1.2.0, at `a1ecc58`, carries them as `verdict_meaning`: direct scope, with embedded components, availability, maintenance and security consequence marked *not evaluated*. Evidence-bearing values for the last four are not yet produced.

5.2 Typed scope

An event e has a scope $S(e) = (E, \chi, \beta)$: a finite set E of typed elements, a completeness state χ, and the basis β of that state (Section 5.2, *Completeness*). Each element has one kind, and the denotation $[\![x]\!]$ of an element is the set of dependency references it covers:
project p: every reference that resolves to project p. It is always an envelope (below): it gives a contextual tie, never a positive, and is never used for exclusion.
repository r: every reference to repository r, whatever its tag or digest. Repository-wide changes such as relocation and withdrawal use this kind.
namespace g/n/∗: every repository under namespace n of registry g. Catalogue-wide changes use this kind, for example `docker.io/bitnami/*`, instead of listing example images that would wrongly be read as the complete scope.
tag r:t: references to r that carry exactly tag t. In place of a literal tag, an element may carry a tag predicate: `<version>` denotes every version-like tag (`1.25`, `7.2.4-debian-12-r3`, not `latest`), and `*` every tag. A predicate applies to a repository or a namespace, so `docker.io/bitnami/*:<version>` is every versioned image under the namespace. A reference with no tag is read as `latest`.
release line (p, ℓ)*:* versions of project p inside line ℓ (`4.2` covers `4.2.11`, not `4.20.1`).
object r@h: the content object with digest h, together with its OCI level (index, manifest or layer).
package (eco, n)*:* every reference to package n in ecosystem *eco*.
registry g: every reference served by registry g, for a registry-wide change such as a service withdrawal or an authentication change.
Each element also has a *role*. An *exact* element claims $[\![x]\!] \subseteq A_e$: everything it denotes is in scope.

An *envelope* element claims only that it helps cover A_e. It may include unaffected references, so it can justify an exclusion but not a positive. Coverage and exactness are different premises (Section 5.4). One completeness declaration justifies negatives and says nothing about positives. The Bitnami withdrawal shows why. The versioned images moved, but `latest` stayed in the mainline. The namespace `docker.io/bitnami/*` is therefore a correct envelope, and an inexact one, since it contains `bitnami/redis:latest`. An exact representation pairs that envelope with an exact element for the versioned tags under the namespace, `docker.io/bitnami/*:<version>`. A tag element alone would not remove `latest` from the namespace union. Roles follow from an element's form. A namespace without a tag predicate is an envelope; every other element, including a namespace with a predicate, is exact. OpenPulse's named `affected_artifacts` are elements of the same set E. They are repository elements, or object elements when they pin a digest. Project-wide scope is an explicit project element, so $E = \emptyset$ means an event that names nothing, not "the whole project".

Completeness. $\chi \in \{\text{verified}, \text{declared}, \text{partial}\}$. *Verified*: the acquisition and scope-construction procedure shows that E covers the affected set, under stated assumptions traceable to the exact responses and pagination state. *Declared*: a producer asserts it. *Partial*: not established. A deployment policy chooses the admissible states $\mathcal{A}$ for negatives: $\{\text{verified}\}$ for high assurance, or $\{\text{verified}, \text{declared}\}$ under a labelled weaker policy. In OpenPulse completeness is opt-in. An unspecified scope is partial. Analysts declare completeness only where the scope is the event itself, such as one release line reaching end of life or one repository's distribution change. Curated events declare it explicitly. No verifier exists yet, so the implemented policy is $\mathcal{A} = \{\text{declared}\}$.

A well-formed declaration is not an independently justified one. A declaration is an assertion. Its only support is the evidence behind the event, and a wrong curated event, a misread announcement or a compromised producer can declare an incomplete scope complete. Then (P1) fails and a negative can be a false clear (Section 5.4). OpenPulse cannot distinguish a declaration from evidence for it, but it records what the declaration rests on. The specification therefore treats the basis as a first-class security input. Declared completeness splits into two bases that carry different evidence. A *definitional* scope is complete because it is the event itself: one release line reaching end of life, or one repository's distribution change. An *announced* scope is complete because a named source says so. A declaration records its source, the scope definition, an evidence reference, the observation time and the validation method, if any. The specification requires the contract to carry χ and that record next to every negative. At `8a9c3a8` each scope records its basis (`definitional`, `announced`, `corroborated` or `verified`), and a declaration without one never clears. The remaining record fields are specified, and the v1.1 contract does not yet carry them (Section 7). Contract 1.2.0, at `a1ecc58`, carries the completeness state, its basis and whether it admits a negative; the source, evidence reference and validation method are still not carried.

The admissibility condition is then machine-checkable. A negative is admissible iff every scope element is decisively excluded and the pair (χ, β) is in the consumer's admissible set $\mathcal{A}$, a set of (state, basis) pairs. Table 9 gives each state's evidence requirement, and what a consumer who admits it is trusting. Policies are combinations of rows. *High assurance* admits only verified scopes, so it issues no negatives today. *Definitional* admits verified and definitional scopes. *Implemented* also admits announced scopes, which is the behaviour at `8a9c3a8`. The bases are recorded and told apart there, but the implementation admits every recorded basis, so a consumer cannot yet select a stricter set. The evaluation reports error rates separately per basis rather than pooling them (Section 10).

Completeness is never derived from a listing. Could a compromised or misbehaving registry serve a partial tag listing, or a truncated pagination state, so that OpenPulse declares

State (basis)	Evidence required	Consumer who admits it trusts	Supports a negative at `8a9c3a8`
Verified	A procedure binding E to the exact responses and pagination state	The procedure and the platform's state assertions	Yes if labelled, but no verifier checks the label
Declared (definitional)	The event's own definition; one release line or one repository	That the event is correctly typed	Yes
Declared (announced, corroborated)	The announcement, plus an independent source family stating the same extent	Two independent sources, and our reading of them	Yes
Declared (announced)	A named source's announcement, with an evidence reference	That source, and our reading of it	Yes
Partial	None	–	Never: exclusion from known members is not a clearance (D4)

Table 9: Completeness states, the evidence each requires, and what a consumer admitting it trusts. Corroboration counts source families, not names (Section 4). If two sources disagree on the extent of a scope, the scope is partial until the conflict is resolved. OpenPulse detects a conflict within one event's evidence (a *contradicts* relation), not between separate events.

a scope complete that is not, and thereby manufacture a false clear? The claim we can support is narrower than "it cannot". It is a dataflow invariant we checked in the code. At `8a9c3a8`, completeness is declared at seven analyst sites in `analyzers/change_analyst.py` and in curated fixtures. Every analyst site builds E from the analysed subject itself: the release line, version or repository that was probed. None builds it from the members of an enumeration. So the *size* of a listing never enters E or χ. A manipulated listing can still change *whether* an event exists. That can cause a missed detection, or a false impact (a phantom removal, if the partiality goes undetected). And because a missing event drops out of the cross-event summary, it can change which events a summary is taken over. What we have not shown is an end-to-end argument that listing manipulation cannot alter event identity, scope construction, declaration choice or the loaded event set by some other route. The invariant is not yet a regression test. Same-host pagination, the page cap, the v2 fallback and the `truncated` flag reduce particular acquisition failures. They do not prove that a listing is complete, or that its sources are independent. A listing that is incomplete in a way the walk cannot detect remains a threat to discovery (Table 18). A future *verified* state would have to bind completeness to the exact responses and pagination state, and would inherit exactly this threat.

5.3 Membership tests

A test $t(d, x)$ returns M (member, with a match level), N (decisively excluded), ? (undetermined), P (project tie) or 0 (no association):

Project. P if d resolves to p. 0 if d resolves with Verified identity to another project. ? if d's identity is unresolved, including a reference that resolves only to itself, whatever its name: a mirror, fork or rename can be called anything. `github.com/etcd-io/etcd` against etcd is one such case.

Repository. M if d names r, whatever its tag or digest. Otherwise N.

Namespace. M if d's registry and namespace are g and n. Otherwise N.

Tag. M if d is $r{:}t$. N if d is $r{:}t'$ with $t' \neq t$, or another repository. ? if d pins a digest of r,

because a tag can move.
Release line. 0 if d resolves through a curated mapping to a project other than p, and ? if its identity is unresolved or the resolution is untrusted. Otherwise compare versions: inside gives M, outside gives N, not comparable gives ?.
Object. Equal digests give M. For unequal digests the answer depends on the levels. If x is an index and d pins one of its children, then M if the event covers *all* children (or the deployment-selected platform's child), otherwise ?. If x is a manifest and d pins the parent index, the result is ?: the deployment's platform selection decides whether the event's platform is used. N only if both levels are known and neither contains the other. If either level is unknown, ?.
Package, registry. Equal package names give M. Unequal names give ?, because names are not canonicalised, or 0 if d is curated as another project. (At `a1ecc58`, names are compared in each ecosystem's canonical form, and a value written as a purl decides by ecosystem and name: F1–F3, Table 1.) The same registry gives M for an exact registry element and ? for an envelope; a different registry gives N. The implementation treats every registry element as an envelope.
Envelopes. Membership in any envelope element gives ? (P for a project element) instead of M. Non-membership is still N. A tag-restricted element gives ? for a reference that pins only a digest, since its tag is unknown. An event's named `affected_artifacts` illustrate a change rather than bound it. So a named literal tag gives M for that tag and ? for the repository's other tags, and 0 for other repositories, while a literal tag in the scope gives N for them. Named and scope elements are evaluated together, and the strongest result wins.
Across reference kinds. A dependency curated as another project gives 0 against every element of the event's project, whatever the kinds. That is positive evidence of no association, and it needs no complete scope. A package reference gives 0 against container-image, object and registry elements, since a package is never one of those. It gives ? against a source-repository element, because a Go module path may name the very repository. (At `a1ecc58`, a Go module at or below a named repository path, directly or by the `gopkg.in` and `golang.org/x` conventions, gives M; positive only, since a vanity path may name the same repository: D-R1, Section 11.) Any other comparison across kinds gives ?. This rule is applied first. The element-specific rules above, including N for an unequal repository, apply only when the kinds agree.

Algorithm 1 has three layers, and the executable oracle must implement all of them. Conformance of the second and third layers alone would validate a weaker kernel than the policy this report states. Layer 1 decides event admissibility. A withdrawn or superseded event is not evaluated. A stale or contested event may still associate a dependency, but it may never clear one. Layer 1 never changes a positive. Layer 2 is the per-element tests, on canonical references. Layer 3 is per-event aggregation, followed by the cross-event summary below. At `8a9c3a8`, layer 1 drops withdrawn and superseded events, withholds negatives from events contested by their own evidence, and applies the freshness bound when one is given.

Across events. d is *affected* if any event affects it, *cleared* only if every event with an association returns NotAffected, and *unresolved* otherwise. Events returning Unknown carry no association and are left out. In the specification that is safe, because 0 requires a Verified resolution to another project, and an unresolved identity yields ? and therefore Related. The implementation used to detect unresolved identity only by name, and that was defect D10 (Table 13). With the etcd relocation and Bitnami events loaded, an unmapped `docker.io/zzz/kvstore` was summarised as NotAffected, while `docker.io/zzz/etcd` was Related. In that reproduction no pair verdict was NotAffected, but the user-facing consequence was the same as a false clear. A second reproduction of D10 was a pair-level false clear: under a version-scoped event, the unmapped reference was excluded on identity grounds (Appendix B). Since `7c0b9d9`, every unmapped reference yields ?.

Algorithm 1 Applicability of event e to dependency d

```
1: function Applicability(d, e)
2:     if e is withdrawn or superseded then
3:         return Unknown                                   ▷ layer 1: not evaluated (N4)
4:     end if
5:     (E, χ, β) ← S(e);    T ← { t(d, x) : x ∈ E }        ▷ every element, no early return
6:     if M ∈ T then
7:         return Affects(strongest member level)
8:     end if
9:     if ? ∈ T then
10:        return Related
11:    end if
12:    if N ∈ T then
13:        if T ⊆ {N, 0} ∧ (χ, β) ∈ A ∧ e is fresh and uncontested then
14:            return NotAffected                           ▷ layer 1 for negatives (N4, N5)
15:        end if
16:        return Related          ▷ exclusion from a scope not known to be complete
17:    end if
18:    if P ∈ T then
19:        return AffectsProject
20:    end if
21:    return Unknown                                       ▷ T ⊆ {0}, including E = ∅
22: end function
```

Cross-event aggregation as a separate property. Per-event soundness does not imply a sound summary: D3 was a summary that displayed a clearance while another event was unresolved. The summary must keep three outcomes apart. *No association exists* (Unknown, from 0). *An association was not established* (Related or AffectsProject, from ? or P). *Decisively excluded* (NotAffected). Table 10 is the truth table over the set R of per-event results. The property is that the summary is *cleared* only if $R \subseteq \{\text{NotAffected}, \text{Unknown}\}$ and $\text{NotAffected} \in R$. The implementation's summary function takes a maximum over seven distinct ranks and then filters a set. Its result is therefore order-independent and idempotent: it depends only on which relationships occur, not on their order or multiplicity. There are 127 non-empty such sets. We checked every ordering of each of them (13,699 sequences), and every sequence of one to five results with repetition (19,607), against Table 10, with no mismatch (`papers/tr-2026-01/check_aggregation.py`). With the order-independence argument, this covers inputs of any length. It covers the summary step only. That an unresolved association reaches the summary as Related rather than Unknown is a separate property, which the D10 regression tests pin.

The table needs no further rows for the harder cases, because each of them is decided per event before aggregation. An event with a partial scope, unresolved identity, a contradiction about its extent, or stale evidence gives Related for d. A withdrawn or superseded event is not loaded, and contributes nothing. Two events with conflicting scopes are two per-event results, and the table combines them. The specification therefore guarantees that a negative from one event never erases uncertainty from another. At `8a9c3a8` a stale event is detected only under a freshness bound, and a conflict between separate events is not detected (Section 4.4).

The summary is taken over the *loaded* event set, and that limits what "cleared" can mean. It states that every loaded event that associates with d excludes it. It does not state that no relevant change exists under adequate monitoring coverage: an event that discovery missed, that a manipulated listing suppressed, or whose acquisition failed is simply absent, and the table cannot see an absence. Lemma 1 and Table 10 give much stronger assurance about the first

Per-event results R (any number of events)	Summary
Some AFFECTS* (artifact, version or package)	affected
No AFFECTS*; some RELATED or AFFECTSPROJECT	unresolved
Only NOTAFFECTED and UNKNOWN, at least one NOTAFFECTED	cleared
Only UNKNOWN (including no events)	no association

Table 10: Cross-event aggregation. A clearance never headlines over an unresolved association, whatever the order or number of events.

statement than anything in this report gives about the second. Neither the summary label nor contract 1.2.0 records the loaded set's coverage; the agent interface lists each source with its status, but a failed acquisition does not withdraw a negative derived from events already loaded. A consumer must read a clearance as "excluded by the events we have", and a coverage field that says which sources and event classes were current at evaluation time is needed before a summary can support more.

5.4 Soundness of negative verdicts

Let A_e be the set of dependency references that are truly in scope of the upstream change described by e, at the time of evaluation.

Lemma 1 (Aggregation)**.** *Assume (P1)* coverage*: if* $(\chi, \beta) \in \mathcal{A}$ *then* $A_e \subseteq \bigcup_{x \in E} [\![x]\!]$*; and (P2)* test soundness*:* $t(d, x) \in \{\mathsf{N}, \mathsf{0}\} \Rightarrow d \notin [\![x]\!]$*. If Algorithm 1 returns* NOTAFFECTED *for* d*, then* $d \notin A_e$*.*

Proof. NOTAFFECTED is returned only when $(\chi, \beta) \in \mathcal{A}$ and $t(d, x) \in \{\mathsf{N}, \mathsf{0}\}$ for every $x \in E$. By (P2), $d \notin [\![x]\!]$ for every x. Hence $d \notin \bigcup_x [\![x]\!] \supseteq A_e$ by (P1). □

Coverage makes $\bigcup_x [\![x]\!]$ an over-approximation of A_e, so it justifies negatives only. Positives need a separate premise.

Lemma 2 (Positives)**.** *Assume (P3)* exactness*:* $[\![x]\!] \subseteq A_e$ *for every exact element* x*; and (P4)* membership soundness*:* $t(d, x) = \mathsf{M} \Rightarrow d \in [\![x]\!]$*, where* $t(d, x) = \mathsf{M}$ *only for exact* x*. If Algorithm 1 returns* AFFECTS *for* d*, then* $d \in A_e$*.*

Proof. AFFECTS is returned only when $t(d, x) = \mathsf{M}$ for some element x, which is exact. Then $d \in [\![x]\!] \subseteq A_e$ by (P4) and (P3). □

Neither lemma needs the other's premises. A system can satisfy the negative premises and violate the positive ones. Before roles were implemented, OPENPULSE violated (P3): D7 and the Bitnami `latest` case were false impacts. At `8a9c3a8` envelope membership no longer yields a positive, so (P3) rests on the exact elements being exact. That is a property of the event's curation, which nothing verifies. Satisfying the positive premises would not make the negatives sound. Its negatives are sound only where the negative-admission conditions hold, and they are enforced only with the gaps stated in Section 5.4 (*The* NOTAFFECTED *contract*).

The lemma is elementary, and that is intended. It is a compact correctness argument for the aggregation rule, not the primary evidence of OPENPULSE's security, and it implies no more assurance than its premises carry. The central technical evidence is meant to be conformance (Section 6), which has been run on a generated matrix and on random cases. The research question is not the implication. It is how an implementation establishes (P1) and (P2), and Table 11 decomposes them. (P1) requires a faithful event, the right elements, completeness and currency. (P2) requires correct identity and correct version, tag and object semantics. Rows marked "not established" are the paths by which a false clear can still occur.

Premise component	Established by	At `8a9c3a8`
P1: event is faithful	Evidence gate, authority	Unsupported claims vetoed; a well-sourced wrong event is not detected
P1: elements are the right ones	Gate (artefact classes)	Missing element not detected
P1: completeness	χ	Opt-in declaration; never verified
P1: currency, non-supersession	Layer 1 of Algorithm 1	Withdrawn and superseded events dropped; freshness bound opt-in (Section 4.4)
P2: identity	Resolver, trust status	Heuristics cannot exclude; a wrong VERIFIED mapping is not detected
P2: version semantics	Comparator, range evaluator	Scheme per range; differential for PEP 440, npm, Go and Maven; others untested
P2: tag and object semantics	Matcher	Tag elements and predicates; unequal digests withheld as undetermined
P2: inventory fidelity	Consumer	Outside OPENPULSE: the declared reference is assumed to be what is deployed
P3: exactness (positives)	Element roles	Envelopes never give positives; exactness of exact elements rests on curation
P4: membership soundness	Matcher, canonical form	Docker Hub hosts folded (D9); package-name forms not canonical

Table 11: Decomposition of the lemmas' premises.

Two counterexamples. Both premises have failed in practice, in our own implementation. *(P1) violated:* the Bitnami event's scope listed five example images, and it was read as complete. `docker.io/bitnami/nginx` is in A_e, every element excluded it, and the verdict was NOTAFFECTED (D4). *(P2) violated:* a Bitnami-packaged `postgresql` image was resolved by a namespace heuristic to a project other than PostgreSQL, so the release-line test returned `N` without an established identity. If the image is a build of the release line its tag names, it is in the element's denotation, and the NOTAFFECTED verdict was a false clear (D5). In both cases the aggregation was correct, and the error came from a premise.

End-to-end statement. The property the deployed system can claim is: if OPENPULSE returns NOTAFFECTED(d, e) at time τ, then $d \notin A_e(\tau)$, *provided* that every P1 and P2 row of Table 11 holds for (d, e). P3 and P4 concern positives only. That means a faithful event with the right elements and a correct completeness declaration, current at τ; a correct identity for d; correct version, tag and object semantics; and an inventory reference that is what is actually deployed. The last premise is outside OPENPULSE, which treats the inventory as untrusted input. The claim is no stronger than its weakest row, and the rows marked not established are why this report makes no unconditional claim about negatives.

The NotAffected contract. Collected in one place, a negative verdict for (d, e) is admissible only if all of the following hold: (N1) every element of E gives `N` or `0`, after canonicalising both references; (N2) any exclusion on identity grounds rests on a curated mapping of d to a different project, never on a name or a self-resolution; (N3) $(\chi, \beta) \in \mathcal{A}$, with the declaration's basis recorded; (N4) e is current: fresh under its class's bound, and not corrected, withdrawn or superseded; and (N5) no unresolved contradiction exists about the extent of E. Otherwise the verdict is RELATED or UNKNOWN, never NOTAFFECTED. Each condition is a check on recorded data, so the contract is machine-checkable once the data are recorded. At `8a9c3a8`, (N1) and (N2) are enforced and regression-tested. (N3) is enforced: a declaration without a recorded basis never clears. (N4) is enforced for withdrawn and superseded events, and for freshness only when the caller sets a bound. (N5) is enforced for a contradiction within an event's own evidence, not between separate events. Each enforced part has a regression test. Failure to establish identity is not evidence of non-identity, and (N2) is that principle stated as a check.

Relation	Representation at `8a9c3a8`	May exclude?
Same repository location	Equality of (registry, namespace, name)	Yes, by location
Same content object	Equal digests; OCI level not recorded	Equality only; inequality `?` (interim)
Canonical alias	Curated alias to a project; heuristic namespace rule (REVIEW-REQUIRED)	Curated: yes. Heuristic: no (D5)
Redirect / relocation	Observed path divergence; continuity of control not inferred	No
Downstream distribution, fork	Separate project with a free-text note; no "contains" relation	Only by curated mapping
Successor project	Not represented	No
Namespace reuse	Not detectable: repository IDs not recorded	No
Unresolved	Any unmapped, self-resolved reference: `?`, whatever its name	No

Table 12: Identity relations and whether each may support an exclusion.

5.5 Identity relations

Table 12 lists the relations that applicability needs. Each mapping carries a trust status. Curated mappings may carry a source, a review date and a maintainer, and they change through code review. They have no temporal validity and no revocation, so an outdated mapping can keep supporting negatives until someone edits it. The specification makes a mapping auditable. It carries provenance, a review time, a revalidation interval after which it degrades to REVIEW-REQUIRED, and an explicit revocation that invalidates every verdict that cited it. A negative that rests on a mapping names that mapping in its contract. None of this is implemented.

Policy for unresolved identity. Identity is a typed, time-bounded relation with five states: exact repository identity, verified alias, curated relationship (such as a downstream distribution), unresolved reference, and verified distinct project. Only the last supports an identity-based exclusion. The principle is that such an exclusion needs *positive* evidence that d is a different project, namely a curated, reviewed mapping. Absence of evidence is never enough: neither a dissimilar name, nor absence from alias lists, nor a heuristic rule. Weaker evidence may establish an *association*, with the confidence cap of Table 7, but never an exclusion. The last column of Table 12 applies the principle to each relation. Mirrors, forks, renames, successors and vendor-repackaged images that have no curated mapping are unresolved, and both the specification and the implementation make them `?`. A package-to-repository mapping is a cross-kind comparison and gives `?` unless a curated mapping relates the two (at `a1ecc58`, a Go module reaches the repository it lives in, positively only; Section 5.3). Whether an upstream event reaches a downstream distribution comes from the event (Section 5.1), not from the identity relation. The price is coverage. Every dependency the catalog does not know is now RELATED to events it was previously UNKNOWN or NOTAFFECTED for. The way to win clearances back is curation, not a looser rule. The OpenTofu fork, for example, is curated as its own project, so Terraform end-of-life events exclude it on that evidence. How much coverage the rule costs is a measurement in Section 10.

Namespace reuse. A relocated repository and a new repository created at the old path look the same to a path-based identity. GitHub assigns every repository a stable numeric identifier, and the design records it with every observation, on any platform that exposes one. If the identifier at a path changes, that is an identity discontinuity. It becomes an event of its own, and evidence observed under the old identifier is not carried across to the new one. Earlier evidence remains valid for the time it was observed. This is not implemented.

5.6 Versions, ranges and coverage

Supported inputs. Dependency inventories are read from manifests and lockfiles for PyPI, npm, Maven and Gradle, Go, Cargo, Composer, NuGet and Swift; from CycloneDX and SPDX JSON documents; and from image inventory files. Registries are Docker Hub and OCI v2 registries, and source repositories are on GitHub. Identity is the canonical reference plus curated mappings. At `8a9c3a8`, package names are not normalised per ecosystem, so two spellings of one package can compare unequal. (At `a1ecc58` they are compared in each ecosystem's canonical form, purl values decide by ecosystem and name, and npm scopes are kept: F1–F3, Table 1.) That could produce an unjustified exclusion only for a package element under a complete scope. Until package names are canonicalised, package elements give `?` on inequality rather than `N`, so a spelling difference never clears. A dependency declared only in an unsupported format is not inventoried at all. It receives no verdict, and COMPLETE inspection says nothing about it.

The comparator implements one scheme, *generic*, as a closed grammar: a dotted numeric core, then at most one pre-release marker (`dev`, `a` or `alpha`, `b` or `beta`, `rc` or `c`, `pre` or `preview`) with an optional number, which orders within the marker. It does not implement the ordering rules of any particular ecosystem, such as dotted SemVer pre-release identifiers, PEP 440 post and local releases, Debian revisions or Maven qualifiers. A closed grammar prevents unsupported forms from being misread. It does not show that the forms it accepts are ordered as the range's scheme orders them. The generic comparator was applied to `SEMVER` and `ECOSYSTEM` ranges alike, and under SemVer its marker ranks disagree with the lexical ordering of pre-release identifiers. That was D11 (Appendix B). Each range is now compared under its own scheme. A `SEMVER` range uses SemVer 2.0 precedence. An `ECOSYSTEM` range uses the advisory's ecosystem: the generic grammar for PyPI, which agrees with PEP 440 on the forms it accepts, and SemVer for npm, Go and crates.io. Any other or missing ecosystem compares plain dotted numbers only, and equality reached only by zero padding is undetermined, because Debian orders `1.0` below `1.0.0`. Before the closed grammar, a numeric suffix was folded into the core, which was D8. Distribution suffixes (`-r75`, `+deb12u1`, `-SNAPSHOT`), unrecognised tokens and epochs (`1:2.3`, `1!2.3`) are not comparable, and yield `?`. Purely numeric date versions such as `20240101` are compared numerically, which is correct only when both sides use the same scheme. An OSV range of type `SEMVER` or `ECOSYSTEM` is a union of intervals walked in version order. `GIT` and unknown types, and versions at or past a `limit`, yield `?`. An advisory clears a version only if every affected entry does, and withdrawn advisories assert nothing. For repository inputs, COMPLETE inspection means that every file the supported readers recognise was read at a pinned commit, and it says nothing about unsupported formats. Under PARTIAL or FAILED inspection, an absent dependency is an inspection limit and never a negative.

6 Q2c: Checking the Implementation Against the Specification

We audited the verdict engine and the version evaluator at `a461bc9`. Table 13 lists every defect the audit found at `8a9c3a8`, with its minimal input and its status. Five defects found later, by the retrospective study and its follow-up (D-R1, D-R2 and D-S1–D-S3, one of them a false clear), are reported in Section 11 and Table 1. Fixing every defect found is engineering evidence, not a demonstrated security property: it does not show that no false-clear path remains, and the late discovery of D8 to D10 suggests that some do. Each fixed defect has a regression test that fails on `a461bc9` and passes on `8a9c3a8` (Appendix C, Table 25). The expected values of those tests were written from the specification before the fixes, which is why they failed on the implementation, so they are not derived from it. They are still chosen cases, not a conformance suite. They do not cover every combination of element kind, reference kind, completeness state and identity status, and a suite that does is the first step of Section 10. The audit covers the paths we examined. It does not show that no other path exists.

	Minimal failing input	Old result	Correct	Kind	Fixed
D1	Event names digests d_1, d_2; dependency pins d_2	NOT-AFFECTED	AFFECTS-ARTIFACT	false clear	`4b92f7d`
D2	Range $[1.0, 1.2) \cup [2.0, 2.3)$; version 1.1	outside	inside	false clear	`4b92f7d`
D3	Event A clears, event B RELATED	headline NOT-AFFECTED	unresolved	clearance shown	`4b92f7d`
D4	Scope of unknown completeness; dependency not listed	NOT-AFFECTED	RELATED	false clear	`f0c3b48`
D5	Heuristic identity resolves elsewhere	NOT-AFFECTED	RELATED	false clear	`4b92f7d`
D6	Version 2.5 against `fixed: 1:1.0`	outside	undetermined	false clear	`b5cbbde`
D7	Event names `redis:7.2`; dependency `redis:latest`	AFFECTS-ARTIFACT	RELATED	false impact	`bfb1ee9`
D8	`SEMVER` range `fixed: 1.0.0`; version `1.0.0-1`	outside	inside	false clear	`9d53ce2`
D9	Namespace `docker.io/bitnami/*`, declared complete; dependency `index.docker.io/bitnami/nginx`	NOT-AFFECTED	AFFECTS-ARTIFACT	false clear	`7c0b9d9`
D10	Unmapped `docker.io/zzz/kvstore:3.5`: against an etcd 3.5 end-of-life scope; and summarised with the etcd and Bitnami events	NOT-AFFECTED; headline NOT-AFFECTED	RELATED; unresolved	false clear	`7c0b9d9`
D11	`SEMVER` range `introduced: 1.0.0-beta, fixed: 1.0.1`; version `1.0.0-dev`	outside	inside	false clear	`dabafef`
D12	`docker.io/redis:7.2` against a complete scope naming `docker.io/library/redis`	NOT-AFFECTED	AFFECTS-ARTIFACT	false clear	`784fed8`
D13	Unmapped package against a release-line event	UNKNOWN	RELATED	clearance shown	`784fed8`
D14	Digest pin under a repository element	RELATED	AFFECTS-ARTIFACT	under-positive	`784fed8`
D15	Release-line member whose tag meets a named example digest	RELATED	AFFECTS-VERSION	under-positive	`d4f7d1f`

Table 13: The audit's defect inventory at `8a9c3a8` (later defects: Section 11). D1–D3, D8, D9: implementation failures. D4: a specification gap, since completeness could not be stated and was assumed. D5: also a test-design failure, because four goldens asserted it. D7 over-matched without clearing. D10 treated an unmapped reference's self-resolution as a verified different project. That cleared it per pair under a version scope, and dropped it from summaries. Two further unsafe paths were closed by interim rules and are listed in Table 16, not here: an index digest clearing its own platform manifests, and a self-resolved reference named like the project, which D10's fix generalises.

Regression tests encoded defects. Four golden scenarios had asserted NOTAFFECTED for Bitnami-packaged images (`postgresql`, `kubernetes`, `kafka`, `cert-manager`) against end-of-life events for the upstream release lines those images contain. Their identity rested on a namespace heuristic. Fixing D5 flipped them to RELATED. Regression stability is not a correct security oracle when the expectations themselves are wrong.

Canonicalisation is semantics. Four operations are kept apart. First, parse the full reference. Second, canonicalise its registry and repository identity (case, Docker Hub hosts, `library`). Third, preserve the tag's spelling and the digest's algorithm and value. Fourth, project the reference onto what each element kind compares: the repository for repository and namespace elements, the tag for tag elements, the digest for object elements. Over-normalising creates false membership, and under-normalising creates false exclusion. At `8a9c3a8`, identity uses the canonical repository form, while tag and digest are read from the original reference and keep their spelling.

Specification conformance. An executable oracle (`conformance/oracle.py`) implements Algorithm 1 and the membership tests from this report's text. It imports none of the implementation, and it takes each case's identity as ground truth, so it tests matching and decision policy, not the resolver. A generated matrix crosses 17 scopes (project-wide, named example, repository, tag, both tag predicates, object, namespace envelope and exact namespace, another repository, two release lines, two package lists and two registries), three completeness states and five admissibility states with 17 dependency references. A project-wide event has no completeness to declare, so the two project-wide scopes run only as partial: $(15 \times 3 + 2) \times 5 \times 17 = 3{,}995$ cases, not the full product of 4,335. Each is run through the oracle and through the implementation, with every disagreement classified. Table 14 gives the results. The first run found three implementation defects (D12–D14), of which D12 produced eight unsafe clears. It also found four places where the specification was wrong or incomplete, and the corrected text is the one given above. Algorithm 1's layer-1 guard had made every verdict RELATED for a contested or stale event, though N4 and N5 restrict negatives only (S1). A dependency curated as another project had been given `?` by the cross-kind and package rules (S2). A package had been comparable with container-image elements (S3). And a named example had been able to exclude (S4). After the fixes, 3,947 cases conform. The other 48 share one label difference: for a package of the event's own project, against a registry scope or a project-wide event, the specification gives a project tie and the implementation RELATED. Both are unresolved, and both block a clearance. At `a1ecc58` the matrix has 21 scopes, adding qualified and mixed package values (F3) and an uncatalogued project's package (F4), and 21 dependency references, adding packages with a stated ecosystem and a package whose name is not its project's slug (F2): $(19 \times 3 + 2) \times 5 \times 21 = 6{,}195$ cases. Of these, 6,123 conform and 72 show the same label difference; none is unsafe.

A hand-designed matrix tests chosen cases, so we added three further checks. *Randomised differential testing* (`conformance/randomized.py`) generates seeded cases that combine everything at once. Each has one to three events per dependency, from two projects; each event has one to four scope elements of mixed kinds, named examples, and random completeness, basis and admissibility; and the dependency is a random image or package, of either project, of a curated third project, or unmapped. Both the per-event verdicts and the cross-event summary are compared. Its first run found one more implementation defect. D15: when a named example left a reference undetermined, the event's scope was never evaluated, so a release-line member was only RELATED. It also found that the image path still made an identity exclusion for an image curated as another project, which needs a complete scope, where S2 gives no association. After both fixes, all 100,000 cases of a seed-7 run agree, in the summary and in all 200,272 per-event verdicts. At `a1ecc58`, with the generator extended for F2–F4, the seed-7 run again agrees on all 100,000 summaries and all 200,070 per-event verdicts. *Defect re-injection* (`conformance/mutation.py`) tests the tests. It patches the running code to re-introduce each historical defect (D1, D3, D4, D7–D15) or to disable each enforced rule (N3, N4, N5, element roles, the package-name rule, S2): 19 single-function mutants at `8a9c3a8`, and 22 at `a1ecc58` with F2–F4 added. Every mutant is caught, and every one without help from the regression tests: by the matrix, by the randomised cases, or, for the comparator mutants D8 and D11, by the version differential below (Table 15). D2, D5 and D6 are not re-injected, because none is a single-function change of the current code. The 22 are therefore historical-defect and rule-disabling mutants, not 22 distinct historical defects. Catching them shows that the harness is sensitive to these chosen perturbations; it does not establish broad fault coverage. Finally, a *build gate* (`tests/test_conformance_gate.py`) runs the matrix, 5,000 random cases and the version differential in the ordinary test suite, in about three seconds. It fails the build on any disagreement beyond the documented label, so conformance is enforced continuously rather than measured once. The comparator was tested differentially against `packaging`, PyPA's reference implementation of PEP 440. Over 233,289 ordered pairs, it agrees on all 224,676 it decides, and it abstains on every pair `packaging` rejects. OSV ranges for PyPI agree on all 7,110 decided

Severity of disagreement (implementation vs oracle)	First run	After fixes	At `a1ecc58`
Matrix cases	3,995	3,995	6,195
Conforming cases	3,350	3,947	6,123
Unsafe: clears, or drops an association	310	0	0
False impact	250	0	0
Under-positive (a member left unresolved)	25	0	0
Over-cautious	0	0	0
Label (both unresolved, or both non-blocking)	60	48	72
Randomised, seed 7: summaries agreeing	–	100,000 / 100,000	100,000 / 100,000
Randomised, seed 7: per-event verdicts agreeing	–	200,272 / 200,272	200,070 / 200,070

Table 14: Specification conformance: the matrix and randomised differential testing. The first run used the implementation before D12–D15 and the specification before S1–S4; most of its disagreements came from S1. "After fixes" is at `8a9c3a8`. The last column is the extended matrix and generator at `a1ecc58`.

cases. SemVer passes the specification's own precedence vectors, both directly and through OSV `SEMVER` ranges. No independent SemVer library was available at `8a9c3a8`. At `a1ecc58`, CI cross-checks the comparator against the pinned `node-semver` 7.8.5, `golang.org/x/mod` v0.42.0 and `maven-artifact` 3.9.16 (`evaluation/comparators/`): all 2,862 npm and 1,892 Go ordered pairs agree, and for Maven it decides 230 pairs, all agreeing, and abstains on 2,422; there are no disagreements. The matrix covers chosen dimensions, not every input. Its oracle was written by the same team, so an external re-implementation would strengthen the result. Agreement of any strength shows only that the implementation follows the decision policy. Because the oracle takes identity, scope and completeness as given, it says nothing about whether the inputs that admit a negative are true of the real change: whether the event describes it faithfully, its scope covers every affected reference, its completeness declaration is justified, its identity mappings are current, and the declared inventory matches what is deployed. Those are the premises of Lemma 1, and their reliability on real events is unmeasured (Section 10).

Defect mechanisms. Appendix B describes each defect's mechanism and repair: D4 in the curated Bitnami data, the tag over-match D7, and D8 to D14; D15 is described in the randomised-testing paragraph above. All are fixed at `8a9c3a8`. Fixing every enumerated defect is engineering evidence, not a demonstrated security property.

Interim conservative rules. Where the full semantics are not yet implemented, fail-closed rules ensure that the gap yields ? rather than a clearance (Table 16). They lower the rate of negative verdicts, and that rate is a quantity to measure in Section 10, not a reason to keep an unsound path.

Cost of resolving OCI levels and tags. Multi-architecture images make the OCI rule matter in practice, so its full resolution is the next implementation step. It need not query manifest lists for every dependency. Resolution is needed only when a digest-pinned dependency meets a digest-bearing element of the *same* repository, which is the only case the interim rule leaves undetermined. Tag-pinned dependencies and digests of other repositories never need it. Each resolution is one manifest fetch by digest. What may be cached indefinitely is immutable content: fetched manifest bytes whose digest has been verified, and the index-to-manifest relation they state. Mutable accessibility may not be cached indefinitely. Whether the registry still serves the object, whether the consumer is authorised to pull it, and what a tag points to all need a time limit. With that split, the content cost is one request per distinct digest ever seen, not one per dependency per run. Recursion into nested indexes is bounded. Tag mutability is a separate cost. Matching tags against scope elements needs no registry call. Resolving a tag to the digest

Mutant	Re-introduces	Matrix	Random	Versions	Tests
D1	First unequal digest excludes	✓	✓		✓
D3	Clearance headlines over an unresolved event		✓		✓
D4, N3	Completeness, or its basis, not required	✓	✓		✓
D7	Tags ignored	✓	✓		✓
Roles	Envelopes give positives	✓	✓		✓
D8	Pre-release numbers ignored			✓	✓
D9	Docker Hub aliases are other registries	✓			✓
D10	Unmapped references unrelated	✓	✓		✓
D11	SemVer ranges use the generic table			✓	✓
D12	`docker.io/<name>` misparsed	✓	✓		✓
D13	Unmapped packages unrelated	✓	✓		✓
D14	Repository element misses digest pins	✓	✓		✓
D15	Named example hides the scope		✓		✓
N4	Withdrawn, superseded or stale events clear	✓	✓		✓
N5	Contested events clear	✓	✓		✓
Package	Name inequality excludes	✓	✓		✓
S2	Curated other project excluded	✓	✓		✓
F2	Project package element ignored	✓	✓		✓
F3	Qualified value matched by name alone	✓	✓		✓
F4	Self-resolved package named like the project is the project		✓		✓

Table 15: Defect re-injection at `a1ecc58`: 22 mutants (two N4 mutants and the D4/N3 pair shown together), all caught, and all caught without the regression tests ("Tests"). The F2–F4 rows were added after the audit snapshot, where the other 19 were caught in the same way. Detectors: the conformance matrix, 5,000 randomised cases, and the version differential against `packaging` and the SemVer vectors.

Rule	Effect	Commit
Epoch syntax → ?	Closes D6	`b5cbbde`
Unknown OCI level on digest inequality → ?	An index digest no longer clears its own platform manifests; no digest-based negatives until levels are resolved	`f0c3b48`
Unspecified completeness → partial	Negatives only from scopes declared complete	`f0c3b48`
Self-resolved reference named like the project → ?	Such references are not dropped as unrelated; generalised to every name by the D10 fix	`f0c3b48`

Table 16: Interim conservative rules.

it currently names is needed only to compare it with a digest element, and that answer expires. It is therefore cached with a time limit and an observation timestamp, and a failed lookup leaves the verdict undetermined. Caching bounds the cost of repeated references. It does not bound the cost of previously unseen digests, of nested indexes, or of registry unavailability, so it is not by itself an argument that the design scales. We have not measured these costs.

OSV conformance evidence. The range tests cover disjoint intervals, out-of-order events, `last_affected`, `introduced: 0`, `GIT` and unknown types, `limit`, mixed entries, withdrawn advisories and epochs. We wrote them from our own reading of the specification, and D6 shows why a differential comparison is needed. It has since been run for PEP 440 and PyPI ranges against `packaging`, and for npm, Go and Maven against the pinned tools (Section 6). For other ecosystems the comparator remains a locally tested approximation of their semantics, not a validated implementation of them. Its safety rests on returning ? for every form it does not recognise.

Required for a negative	Internal computation	Contract 1.2.0	Agent response	At consumption
N1: every element excludes	Per-element results	Match method and free-text reason only	Relationship and reason	No
N2: identity excluded only through a curated mapping	Enforced	`identity_status`, `identity_via`	`identity_status`, `resolved_via`	No
N3: complete scope with recorded basis	Enforced	`scope.completeness` (state, basis, `admits_negative`); absent in 1.1	Per cause, state and basis	No
N4: fresh evidence	Only under a caller's bound (opt-in)	`fetched_at` per source; `freshness_bound_applied`	90-day bound by default; `evaluation_context`; replay mode flagged	No
N4: not withdrawn or superseded	Such events are not loaded	No expiry, supersession or revocation of the document	–	No
N5: uncontested	Within one event only	`relation` per source	Not shown	No
Meaning of the negative	Direct scope only	`verdict_meaning`	`verdict.meaning`	–
Coverage of the loaded event set	Not specified	Not exported	Sources with status	No
Producer authenticity	Not implemented	Signature fields `null`	–	No

Table 17: Assurance of a negative, at `a1ecc58`, with the agent column at `323bb3c`: what the specification requires, where the implementation computes it, what leaves the system, and what is checked when a verdict is consumed. Before `323bb3c` the agent response showed neither completeness nor freshness, and applied no bound.

7 Q2d: Evidence for Consumers

Evidence contract. One JSON document is produced per (dependency, event) pair and validated by a published JSON Schema. Version 1.1 carries the event context, the identity with its trust status, the scope, the match, the assessment, evidence references with authority, the temporal roles (`null` means unknown), the unknowns, a recommendation, the provenance and the inspection coverage. Its `content_hash` is SHA-256 over sorted-key, compact JSON. A document that carries its own unauthenticated hash supports only an *internal-consistency* check. It shows that a document is unchanged relative to a prior state only if the expected hash came from a trusted channel. Signature fields exist and are always `null`. There is no supersession or revocation. The contract is therefore suitable for evidence exchange and human review, not for automated enforcement. That would need producer authenticity, evidence freshness, supersession, revocation, and the completeness state and negative-meaning fields of Section 5.1. Contract 1.2.0, at `a1ecc58`, adds the last two (`verdict_meaning`, `scope.completeness` with its state, basis and `admits_negative`, and an `evaluation_context`); producer authenticity, expiry, supersession and revocation remain absent.

Assurance levels. Three kinds of negative must be kept apart: one the implementation admits by default, one that satisfies N1–N5 in full, and one that is still suitable for a decision when it is consumed. Table 17 shows, for each condition the specification requires of a negative, whether it is computed internally, exported in contract 1.2.0, shown to an agent, and checked when the verdict is consumed. The last column is empty: a consumer can validate a document's shape and its internal hash, but nothing re-checks currency, coverage or revocation at consumption time.

Agent interface. An MCP server offers three read-only tools: status of one dependency, a check of a dependency file, and supported release lines. The server is confined to its root directory and to three hosts. Its response mixes two kinds of field. *Structured* fields come from fixed vocabularies computed by OpenPulse: relationship, affected flag, confidence, identity status, event type and counts. *Free-text* fields embed upstream-derived strings: verdict reasons, which quote references and scope values, evidence URLs, repository names and licence strings. The server returns no repository descriptions, release notes or advisory prose. Even so, references and names are attacker-chosen strings. Free-text fields are not encoded or marked as untrusted, and no injection testing has been done. A deterministic verdict does not make an LLM consumer resistant to injected text in those fields. In the decision-point model of Section 3, the server is an advisory lookup at stage 1 and an inventory check at stage 3. It provides no approval or enforcement at stage 2. Every response carries a fixed note stating that its content is evidence, not a decision, and that Unknown never means safe. At `8a9c3a8` the note does not qualify NotAffected, and an agent sees a bare negative label. At `a1ecc58` each verdict carries `verdict.meaning` from contract 1.2.0, and the note tells the agent to decide from it, so a negative states what was not evaluated. At `a1ecc58` it did not carry the completeness basis, and the agent tools applied no freshness bound, so the interface could issue NotAffected on evidence of any age without saying so. Since `323bb3c` the tools have two explicit modes. *Current* mode, the default and the only one the MCP server uses, evaluates as of now with a maximum evidence age of 90 days, the bound of the retrospective study; a negative from an older event becomes Related with the reason *stale*. *Replay* mode, available only through the Python API, applies no bound, since offline replay evaluates evidence as of a past date; its response marks `freshness_bound_applied` false, lists evidence currency as not established, and adds a warning to the note. Every response now carries an `evaluation_context` (mode, evaluation time, bound) and, for each cause, the scope's completeness state and basis. Contradictions and the coverage of the loaded event set are still not shown, and nothing re-checks a verdict when it is used (Table 17). Results should be reported separately for the two modes. The interface is still an unvalidated integration boundary, not a security mitigation. It becomes one only once the structured and untrusted parts are separated and tested against a reproducible adversarial corpus.

Injection mitigation design (future work). The aim is structural separation, so that an agent can make its decision without ever reading upstream text as part of an instruction-bearing field. We specify four measures, none of them implemented yet. (i) *Decision fields are closed vocabularies.* Relationship, affected flag, confidence, identity status, event class, completeness state and dates are enumerations or validated formats. The tool description tells the agent to decide from these fields alone. (ii) *Upstream strings are quarantined.* Every upstream-derived string moves into a single block labelled `untrusted_upstream`, together with its source and provenance. Strings are length-bounded and stripped of control and markup characters, and identifiers are checked against the charset rules the collectors already enforce. Nothing outside that block contains upstream text. (iii) *Reasons are templated.* Verdict reasons are generated from fixed templates whose slots hold only validated identifiers. They do not quote upstream content. (iv) *Testing.* The agent experiment in Section 10 includes an injection condition, and a corpus of hostile repository names, tags, URLs and licence strings is run against the server in regression tests. Sanitising text cannot make an LLM immune to injection. Labelling is not isolation either. A quarantined block in the same model-visible response still reaches the model's context, and nothing makes the model ignore it. What the design achieves is narrower: the evidence an agent needs never *requires* reading untrusted text. The decisive question is whether hostile content reaches the decision context and changes behaviour, and that is empirical. The agent experiment therefore compares three response forms. The first is structured-only. The second is structured, with raw upstream evidence retrievable only through a separate tool call.

The third is mixed, with the untrusted content labelled. Only the second keeps untrusted text out of the context by default.

8 Threat Model

OpenPulse reports what upstream platforms say about themselves. When an attacker controls a legitimate upstream account, the platform truthfully reports a malicious state, and OpenPulse records it as faithfully as a benign one. Neither a verdict nor the absence of a finding is evidence of trustworthiness.

Three properties are easy to conflate, and OpenPulse establishes only the first. *Observed platform state*: what a platform reported, from one vantage point, at the acquisition time. *Identity continuity*: that the principals or repository behind a name are the same as before. OpenPulse observes path divergence, not control, and does not record repository identifiers. *Publisher trust*: that the content is legitimate. That is outside OpenPulse. Provenance frameworks such as in-toto, SLSA and Sigstore help establish claims about origin and build process, and signing binds content to a key holder. Neither establishes that the published content is benign. The interface emits no trust, safety or reputation field. Each event is typed by what was observed, and the agent note says that output is evidence, not a decision. That labelling does not *prevent* a consumer from promoting observed state into trust, and no schema field could. OpenPulse is an advisory instrument, not a validated guardrail for autonomous adoption. Independent-looking sources may repeat one incorrect announcement. A read-only service still influences what an agent does next. A schema-valid field can carry a wrong conclusion. Signed evidence would authenticate its producer, not establish that it is correct. And a missed discovery removes an event from a summary entirely. Negative-meaning fields would make promotion explicit and checkable. Preventing it belongs to the consumer's policy. Table 18 organises the threats by the property they attack.

Property	Threat	Status	Notes
Upstream information	Compromised legitimate account	out of scope	Recorded faithfully; never read as trust
	Selective visibility (auth, region, rate limit)	partial	Errors and truncation recorded; vantage point not recorded
Upstream identity	Namespace reuse at an old path	not handled	Repository IDs not recorded
	Poisoned curated mapping	partial	Heuristics cannot exclude; curated mappings trusted; no revocation
	Copied sources counted as independent	not handled	Fallback to source name
Freshness	Stale or replayed response	not handled	Local acquisition time only; source timestamps unchecked
	Change during pagination	not handled	Traversal over an interval, not a snapshot
	Silently partial or manipulated listing	partial	Listing size never enters E or χ (checked dataflow invariant); can cause missed detections, phantom removals, and a changed set of summarised events
Local evidence	Accidental or partial corruption	detected	Hash chain
	Suffix rewrite, rollback, backdating	not handled	Unkeyed; no witness
Applicability engine	Implementation defects	partial	Table 13
	Wrong completeness declaration	partial	Basis recorded and required; the declaration itself is trusted (Table 9)
Downstream use	Injection via free-text fields	not handled	Structural separation designed (Section 7), not implemented; no testing
	Negative read as clearance	partial	Meaning carried in contract 1.2.0 and the agent response (`a1ecc58`); a consumer may ignore it
	Older contract substituted for a current one	not handled	Internally consistent; unsigned, no expiry or supersession

Table 18: Threats by property.

9 Q3: Development Case Ledger

Before collecting cases we set an acceptance target: at least five non-lifecycle changes, detected by the pipeline and replayable offline, with any unreproducible candidate retracted. Each case is pinned by a golden test that replays the recorded raw evidence through the production analysts. The authors found the cases while developing the detector, so the ledger shows *detectability*, not effectiveness.

Relocations (Cases 6–9). The detector compares the queried path with the canonical path GitHub returns, and fires when the owner differs. It names the old path as the affected artefact. It observes path divergence. It does not observe a change of controlling principals, which can also happen with no path change. Nor does it show that the old path can be reclaimed [18]. Same-owner renames are silent by policy, which leaves a coverage gap.

Distribution (Cases 2 and 10). Case 2 is treated as a change only because official announcements say versioned images moved. The observation corroborates the announcement. Two different completeness requirements apply. The claim that the mainline has *no* version-like tag needs a complete mainline listing, and the ledger records a full-set observation of its 534 tags. The claim that the legacy namespace *has* versioned tags needs only one example, so the truncated lower bound of $\geq$919 suffices. Removal inference is withheld on truncated sets because removal is a claim about absence. Case 10 rests on tag listings from two endpoints. No

#	Class	Observed change	First det.	Effective (provenance)
2	Distribution	`bitnami/redis`: 534 tags, 0 version-like; `bitnamilegacy/redis`: ≥919 version-like	2026-10-02	2025-08-28 (official notice)
6	Relocation	`fireeye/capa` → `mandiant/capa`	2026-10-06	unknown
7	Relocation	`coreos/etcd` → `etcd-io/etcd`	2026-10-06	unknown
8	Relocation	`containous/ traefik` → `traefik/traefik`	2026-10-07	unknown
9	Relocation, archive	`facebook/ react-native-fbsdk` → `facebookarchive/...`; archived	2026-10-07	unknown (last push 2021-03-26)
10	Distribution	`bitnami/spark`: no tags listed; `bitnamilegacy/spark`: 936 tags	2026-10-07	2025-08-28, postponed to 2025-09-29 (official notices)

Table 19: Development ledger (numbers follow `docs/DISCOVERIES.md`). Detections came from single-day probes. Pipeline confidence is EMERGING throughout.

Reference	Verdict	Observed listing	Consequence	Evidence missing
`bitnami/spark:3.5`	AFFECTS-ARTIFACT	no tags listed	Withdrawal; migrate	Manifest resolution
`bitnamilegacy/spark:3.5`	AFFECTS-ARTIFACT	tag listed	Frozen distribution; no updates	Manifest resolution; whether fixes land
`apache/spark:3.5`	NOT-AFFECTED	not observed	None from this event	– (declared-complete scope)

Table 20: Case 10: membership is not consequence.

reference-specific manifest resolution was performed, so Table 20 reports listings and makes no claim about whether a reference resolves.

Retracted candidates. On 2026-10-01 the pipeline reported tags disappearing from four official images. Three were formalised as Cases 3–5. One mechanism explained all four: the probe read only the five most recent tags, so a tag pushed out of that window looked removed. Every candidate was present in the full listings. The fix added full pagination, a `truncated` flag that withholds diffs, and a parser-version re-baseline.

Lead times. Table 21 applies the definition of Section 4.4. All non-lifecycle detections were late, and the two prospective cases belong to a class that catalogues already record. No advance warning for an unrecorded change class has been observed.

Golden scenarios. Eleven scenarios (Bitnami, iText, MinIO, Django, Redis, PostgreSQL, Kubernetes, Kafka, cert-manager, Grafana, Terraform) fix expected identity and applicability outcomes. They are regression tests, not measurements, and four of them had encoded D5.

Case	Announced	Effective	First det.	Lead (d)	Catalogued
Retrospective: observed historical changes					
2 Bitnami split	2025-07-16	2025-08-28	2026-10-02	−400	no
10 Bitnami Spark	2025-07-16	08-28 or 09-29	2026-10-07	−405 / −373	no
6–9 Relocations	unknown	unknown	2026-10-06/07	undefined	no
Prospective: scheduled lifecycle events					
PostgreSQL 14 EOL	policy page	2026-11-12	2026-10-06	37*	yes
Redis 8.0 EOL	policy page	2026-12-01	2026-10-06	56*	yes

Table 21: Lead times. *Catalogued*: whether an existing lifecycle or vulnerability catalogue already records the change. All first detections came from single-day probes, so there was no monitoring start, polling policy or acquisition-failure record. Effective dates are as cited and were not independently verified. *Relative to the current schedule.

10 Q3: Evaluation Protocol

This section is a draft prospective protocol. Apart from specification conformance (Section 6), the basic cost measurements below and the exploratory retrospective study of Section 11, none of it has been run, and nothing in it is evidence of effectiveness. It fixes the metrics, annotation rules and analysis plan before any data are labelled, which reduces analytical flexibility but does not eliminate it. A registration draft now records the primary endpoint, the numeric margins, the guardrails, the sample-size reasoning and the analysis code (`evaluation/REGISTRATION.md` and `evaluation/registration.py`), written before any held-out label exists; we call its values *pre-specified*. It becomes a registered protocol only when it is archived (OSF Registries or Zenodo) with its registry identifier, freeze commit, event window and annotators filled in, before any held-out label exists. That has not happened yet. Before the evaluation, the evaluated subset must enforce the full negative-admission policy, or abstain wherever it cannot. At `8a9c3a8` that holds with the gaps stated in Section 5.4, and the evaluated configuration sets the freshness bound. The coverage cost of the policy, D10 and the package-name rule in particular, is among the first measurements, and it should be reported as unresolved references per inventory and per monitoring cycle, not only per pair. Expanding this protocol further has diminishing returns. Conformance has been run (Section 6). The next step is the primary applicability comparison, after the registration is archived.

Specification conformance. This step has been run on a generated matrix (Section 6, Table 14). What remains is to extend it to OCI level relations against recorded registry responses, and to have the oracle re-implemented outside the team. The design is as follows. Before measuring effectiveness, test whether the implementation conforms to the specification. Build an executable oracle directly from Algorithm 1 and the membership tests of Section 5.3, independent of the matcher's code. Generate a finite matrix of element kinds × dependency-reference kinds (tag-pinned image, digest-pinned image, source repository, package with and without version) × completeness states × identity statuses. Add adversarial cases: an empty scope, several matching elements, conflicting events, unsupported version forms, unresolved identities, exact and mutable tags (`latest`), multi-tag and repository-wide scopes, and OCI index and manifest relations. Run the same fixtures through the oracle and the implementation, and report *every* discrepancy, not only those already known (the project-tie label difference). For OCI level relations, record real registry responses for multi-platform images and check the directional rules against them. For versions, compare the range evaluator differentially with the ecosystems' own comparators, and with published test corpora where they exist. Cover disjoint ranges, ecosystem ordering, epochs, pre-releases (including numeric identifiers, D8), distribution revisions, withdrawn advisories and unsupported range types. Publish the exact supported subset that results. Conformance is necessary but not sufficient. A matcher can

conform perfectly and still clear a dependency that an incomplete event omitted. The two layers are therefore evaluated separately: conformance asks whether the implementation matches the specification, and the applicability evaluation below asks whether the specification's inputs and premises match real upstream changes and real identities.

Applicability. Use real upstream events whose scopes are adjudicated independently, and held-out (dependency, event) pairs. Include adversarial identity and version cases, and real inventories that contain distributions, mirrors and embedded components. Annotators label every pair, including those OpenPulse calls NotAffected, Unknown or Related. Every method receives the same event evidence and inventory, so that differences in collection coverage do not confound matching quality. Report two clearance error rates. With F the number of truly affected pairs labelled NotAffected, A the number of truly affected pairs, and C the number of NotAffected predictions:

$$\mathrm{FCR}_{\mathrm{affected}} = \frac{F}{A}, \qquad \mathrm{FCR}_{\mathrm{clear}} = \frac{F}{C}.$$

The first measures affected cases lost to clearance, and the second measures how unreliable an issued clearance is. $\mathrm{FCR}_{\mathrm{affected}}$ has the same denominator for every method, so it is the comparable one. $\mathrm{FCR}_{\mathrm{clear}}$ is conditioned on each method's own clearances, so methods that clear different subsets are compared on it only together with a paired analysis on the pairs that both methods clear. When $C = 0$, $\mathrm{FCR}_{\mathrm{clear}}$ is undefined and is reported as such, never as zero. When no false clear is observed, pairs are not treated as independent trials, because many pairs share an event. The primary uncertainty is stated over independent events and projects. With n_e events and no event containing a false clear, the one-sided 95% upper bound on the per-event failure probability is about $3/n_e$. A pair-level Clopper–Pearson bound ($\approx 3/A$ or $3/C$) may be reported, but only as a descriptive figure, labelled with its independence assumption. Resampling data that contain no error cannot reveal unseen failure modes. So a zero result is reported together with the number of independent events behind it. Report them alongside precision and recall of *affected*, the false-impact rate, and decisive coverage:

$$\text{coverage} = \frac{\#\{\text{Affects through an exact element}\} + \#\{\text{admissible NotAffected}\}}{\#\{\text{evaluated pairs}\}}.$$

Unknown (no association), AffectsProject and Related (contextual or unresolved), and abstention for stale or unavailable evidence are reported separately and never counted as decisive. Otherwise a method could raise coverage by producing more uninformative Unknown results. Report clustered bootstrap intervals by event, stratified by event class, match type, evidence strength, identity status, completeness state and basis, and confidence level. Report the cost of conservative behaviour: Unknown and Related rates by event class, identity status, completeness state and reference type, the review burden they imply, and the share of unresolved pairs that adjudication finds truly affected.

These metrics must be read together. A low false-clear rate is not enough if most hard cases become Unknown or Related. High decisive coverage is worthless if it comes from unsound exclusions. Decisive coverage measures how often an answer is specific, not whether it is right. We therefore report the full error–coverage trade-off: one operating point per deployment policy (the admissible set $\mathcal{A}$ of Table 9, and a minimum verdict confidence). A deployment selects the point whose false-clear rate it can accept, and reads off the coverage it pays for it.

Unit of analysis and annotation. The unit is a (dependency, event) pair. Pairs that share an event, or an underlying project, are correlated. Intervals are therefore clustered by event, and a sensitivity analysis clusters by project. Thousands of pairs derived from a handful of events are not thousands of independent scenarios. Every result therefore reports the number of

independent events and projects next to the pair count, with event-level and per-class results. The number of events is justified by a power calculation, registered before labelling, for the primary comparison. Two annotators label each pair as in scope, out of scope or undeterminable from the evidence. They use the event's cited sources and the registry or repository state frozen at a recorded date, and they are blind to every method's output. Published guidelines set the precedence when sources disagree. The announcing source defines the intended scope. Registry and repository observations establish current state. A conflict between them is labelled *ambiguous* and reported as such, not forced into a binary label. A third annotator adjudicates disagreements, and agreement is reported as Cohen's κ. Pairs that remain undeterminable or ambiguous are not counted as false clears or false impacts. They are not ignored either: excluding them would reward a method that answers confidently where the evidence is genuinely unresolved. They feed a separate *unsupported-decisiveness* rate, the share of such pairs given an unqualified AFFECTS or NOTAFFECTED. Abstaining on them is correct.

Decision criteria. The hypotheses are fixed before labelling. There is one primary endpoint, at one operating point: the implemented policy $\mathcal{A}$, with no minimum confidence. The registration draft compares OPENPULSE with the minimal conservative reference (iv), which never clears. Earlier versions of this protocol made (v) primary; the registration draft discloses the change. The claim under test concerns the model, and (v) shares that model, so OPENPULSE against (v) measures only the engine; the change is disclosed in the registration. Two primary hypotheses must both hold. *Safety (H1):* the exact one-sided 95% Clopper–Pearson upper bound on the share of events in which OPENPULSE issues a false clear, minus that share for (iv), is at most $\delta = 0.05$; H1 is decidable only with at least 60 events at risk, and with fewer the result is "not decidable", never a pass. *Usefulness (H2):* the paired difference in resolution, OPENPULSE minus (iv), is at least 0.10, with an event-clustered bootstrap one-sided 95% lower bound above zero (4,000 resamples, fixed seed). At 60 events, one false-clear event already fails H1 (bound 0.077). Secondary tests are Holm-corrected and never rescue a failed primary result: equivalence of OPENPULSE and (v) within ± 0.05 by two one-sided tests (S1), and the false-clear excess of the naive baseline (S2). Three guardrails are reported beside the verdict without changing it: silent misses at most 0.02 of in-scope pairs (G1), OPENPULSE's unresolved rate at most (iv)'s plus 0.05 (G2), and every class with an unresolved rate above 0.60 flagged (G3). The registration states which of these values came from judgement. A result that improves coverage only by raising the false-clear rate beyond the margin counts as a failure.

Baselines. The baselines are: (i) direct metadata rules; (ii) namespace-aware matching without completeness or identity caps; (iii) curated feeds with SCA matching; (iv) a *minimal conservative reference*; and (v) a *strong conservative baseline*. Beating (ii) is expected by construction. The minimal reference (iv) is the smallest implementation of the principle. It matches exact repository and package names only, with no aliases or identity mappings. It admits a negative only from a scope in the same $\mathcal{A}$. It returns RELATED for every version-scoped event unless the exact version string is listed. That makes coverage gains on versions easy, which is why (v) is compared separately (S1); (iv) is nonetheless the registration draft's primary comparator, for the reason given under the decision criteria. The strong baseline (v) uses the same scope language and admissibility policy, exact identities, the ecosystems' own established version comparators, and conservative handling of every unsupported relation. Both methods receive the same typed scopes, the same curated mappings, the same event-confidence and freshness policies, and the same unknown and contradiction information. The completeness condition, the identity rule and the admission policy are therefore *shared*, and (v) cannot measure their value; it measures only what OPENPULSE's matching code adds beyond them. That residue is small and can be stated in advance. (v) relates a reference to an element only through the same reference kind, a curated mapping, or the project element; OPENPULSE additionally relates a Go module path to

the source repository it names (D-R1), and reads the release from an image tag with a variant suffix (`node:12-alpine` as release 12), where (v) compares image tags only as plain dotted numbers. Pairs that exercise neither are expected to receive the same verdict from both. (v) is therefore the comparison that tests whether the engine adds anything (S1 above); the primary comparison is with (iv), for the reason given there. The comparison therefore evaluates matching and decision policy, not the whole intelligence pipeline, and its conclusion is stated that way. The ecosystem comparators are now pinned: `packaging` (hash-locked) for PEP 440, `node-semver` 7.8.5 for npm, `golang.org/x/mod` v0.42.0 for Go modules and Maven's `ComparableVersion` from `maven-artifact` 3.9.16 (jar SHA-256 checked). Each runs through a bridge that reports its library version, and the analysis refuses to run unless every pin matches; CI validates the bridges against the libraries' documented semantics. `dpkg` ordering is out of scope, and Debian versions remain undetermined. Ablate the gate, the identity cap, completeness and the coverage qualifier.

Population. Events are sampled from sources that do not feed OpenPulse: vendor and project announcements, distribution changelogs, and platform transfer and archival records, over an observation window that starts after the implementation freeze. Inclusion criteria are fixed per event class before sampling. Inventories come from collaborators and public projects that contain container images, distributions and embedded components. Pairs are drawn by stratified random sampling over event class and reference kind, so that the sample is not limited to events the detector recognises easily. Every pair is labelled, including those OpenPulse never associates.

Discovery. Monitor a population fixed in advance, at a stated polling interval and from a recorded start date. For each event, record when it first became publicly observable, when OpenPulse detected it, when it was adjudicated, and when a conventional dependency-security workflow would have learned of it. End-to-end decision latency is the time from observability to a correct, actionable verdict for an affected pair. Build an independent reference set from sources that do not feed OpenPulse: vendor notices, distribution changelogs and manual review of a random sample. Report detections per event class, recall bounds limited to the reference set's coverage, and leads with monitoring context, as lead-time distributions rather than single values. Report four outcomes separately and never combine them into one lead-time claim: detection of a change already documented elsewhere; earlier detection of a change not yet in a relevant catalogue; discovery of a change that existing tools do not represent; and improved applicability of an already known change. Only the middle two measure additional discovery value. Measured recall is over the reference set; it is not an estimate over all upstream changes. Events missed by both OpenPulse and the reference set cannot be counted. The overlap between the two gives a capture–recapture estimate of their number. We report it only as an indication, because its interpretation assumes independent sources and a homogeneous population, and neither is likely to hold. Confidence categories are evaluated against observed correctness, as an error rate per category. Source independence is audited on a sample of corroborated events by tracing each source family to its origin, including syndicated announcements and mirrored advisories. Today there is no central registry of source families: a family is declared per source, and `derived_from` chains are folded (Section 4). The registry is to be published with the audit.

Agent decision experiment. Compare three main conditions: an agent with no external dependency evidence; with conventional vulnerability and lifecycle evidence; and with OpenPulse evidence added. Within the third, include tasks whose event scopes are deliberately incomplete or in conflict, so that a correct agent must abstain or ask for review. Use tasks whose candidate dependencies differ in archival, namespace, distribution or release-line status. Measure the dependency choices, unsupported safety claims, requests for human review, be-

haviour when evidence is unresolved, and task completion. Freeze the evidence snapshot, tool permissions, prompts and model versions. Include an injection condition in which malicious upstream prose appears in the same response as valid structured evidence. The outcome is whether decisions move in the intended direction without excessive abstention or task failure, not whether the agent's wording changes. A successful AI-assisted decision is a choice that matches the adjudicated evidence and is reached without an unsupported safety claim, not merely a generated contract. Count separately incorrect dependency choices, unsupported safety claims and inappropriate clearances. Abstention is *beneficial* when the adjudicated evidence is unresolved, and an *unnecessary refusal* when the evidence is decisive. A decisive choice made where the evidence is unresolved is *unsupported decisiveness*, even when it happens to be right. Each task's labels predefine the set of acceptable dependencies, not a single right answer, so that a technically acceptable alternative is not scored as an error. The injection condition uses malicious repository names, tags, URLs and licence strings, and it is scored by whether the decision changes, not by what the agent writes. Three failure modes are scored separately: following hostile text, misreading a legitimate verdict, and acting on insufficient evidence. All conditions use the same model, task, tool permissions and decision instructions, and only the evidence source changes. A fourth arm gives the agent OpenPulse's tool descriptions and instructions with the evidence withheld, so that the effect of the evidence can be separated from the effect of clearer instructions. Decision labels are predefined from the adjudicated evidence. Results are reported per model and per task, not as one aggregate success rate.

Cost and scale. Measure along five dimensions: inventory size, number of loaded events, scope size, registry tag count and history length. Report separately the cost of offline matching and the cost of upstream acquisition. Use small and large representative inventories, cache-cold and cache-warm runs, and injected failures. Compare candidate filtering by index with all-pairs evaluation, and report rate-limit gaps and recovery. Report matching time, memory, evidence-store growth, request volume per platform, cache hit rates and freshness-driven re-fetches. For agent use, report latency with a warm and a cold cache, behaviour when evidence is unavailable, rate-limit stalls, evidence expiry and refresh cost, and whether collection can stay outside the interactive request path without making evidence too stale to use.

Today's acquisition behaviour is the baseline for those measurements. Collectors classify HTTP 429 responses as retryable rate-limit errors and record them as acquisition gaps, never as results. There is no automatic backoff or request scheduling. Docker Hub tag walks are capped at 30 pages of 100 tags and fall back to the registry v2 listing, because the Hub refuses anonymous pagination beyond an offset of roughly 500. GitHub requests use a token when one is provided. The agent interface caches lookups per process for one hour. Continuous monitoring of thousands of repositories in CI will need scheduling, conditional requests and backoff that do not yet exist. How request volume and gap rate grow with the monitored population is a result the evaluation must produce, not one we can assume.

Basic cost measurements. The basic measurements have been run, on one machine (Windows 11, 12 logical CPUs, Python 3.14) with the production engine and store (`benchmarks/cost.py`; Table 22). Matching is linear in the number of dependency–event pairs. It cost about 42 µs per pair at `8a9c3a8`, rose to about 75 µs at `a1ecc58`, and is about 25 µs with the catalogue-map cache (`0174a26`) at 100 and 1,000 dependencies. At 5,000 dependencies the cached runs varied most (24–43 µs per pair; median 1.9 s for 10 events, against 2.1 s and 3.8 s), with a peak of 25 MiB throughout. The rise was a fixed overhead per package dependency, not per scope element: since F1, resolving a package's identity rebuilt the catalogue's package and alias maps on every call, which took about three quarters of the matching time in a profile. Memoising those maps per catalogue object removes it; every decision is unchanged (the matrix, the 100,000 random cases, the re-injected mutants, and the development and retrospective case

Measurement	Setting	At `8a9c3a8`	At `a1ecc58`	With cache
Matching, by inventory size (10 events)	100 / 1,000 / 5,000 deps	0.04 / 0.41 / 2.06 s	0.07 / 0.75 / 3.78 s	0.02 / 0.25 / 1.87 s
Matching, by event count (1,000 deps)	1 / 10 / 100 events	0.07 / 0.41 / 3.87 s	0.14 / 0.76 / 6.85 s	0.04 / 0.25 / 2.66 s
Matching, by scope size (1,000 deps, 1 event)	1 / 100 / 1,000 elements	0.05 / 0.32 / 2.61 s	0.14 / 0.46 / 3.29 s	0.05 / 0.42 / 3.28 s
Peak memory	5,000 deps × 10 events	24.6 MiB	24.6 MiB	24.6 MiB
Sweep, per repository (51 tags)	median of runs 2–20	35 ms	34 ms	35–45 ms
Store growth, per repository per run	steady / one new tag per run	6.1 / 7.1 KB	6.1 / 7.1 KB	6.1 / 7.1 KB
Agent lookup, cold	5 packages	0.5–2.5 s, 2–3 requests	not repeated	not repeated
Agent lookup, warm	same packages	2–5 ms, 0 requests	not repeated	not repeated

Table 22: Basic cost measurements (`benchmarks/REPORT.md`). One machine; medians of three runs for matching. The `a1ecc58` column is the median of three runs interleaved with two fresh runs of `8a9c3a8` on the same machine, which reproduced the earlier `8a9c3a8` figures within about 15%. *With cache* is `a1ecc58` plus the catalogue-map cache (`0174a26`), three runs on the same machine. The sweep code is unchanged across the three columns; its spread is run-to-run variation. Agent lookups are live and were not repeated.

files). Scope size is the expensive dimension. Every element is evaluated for every dependency, with no index, which costs about 2.6 µs per element per pair at `8a9c3a8` and about 3 µs at `a1ecc58`, so a 1,000-element scope costs about 3 ms per dependency. Namespace elements keep real scopes small (the Bitnami event has four), and an index from repository to elements is the obvious remedy. The observation store writes a full snapshot on every sweep, about 6–7 KB per repository per run at 51 tags, even when nothing changes. Each sweep also reads the repository's whole history, so its cost, about 35 ms per repository, rises slowly with history length. Daily sweeps would therefore accumulate about 2 MB per repository per year, and a growing sweep time, until snapshots are replaced by deltas or history is compacted. Through the agent interface, a cold lookup made 2–3 HTTP requests and took 0.5–0.8 s (2.5 s for the first, including connection setup). A warm lookup took 2–5 ms and made no request. One session saw no errors and no rate limiting. At 1–3 requests per package, an unauthenticated GitHub client allows 20–60 cold lookups an hour. Not measured: large tag walks, behaviour under rate limiting, monitoring of a large population, and indexed matching. The absolute numbers depend on the machine and the network; the shapes of the curves are the result.

Leakage control and order. Freeze the implementation at a tag before labelling, together with the event definitions and matching rules. Freeze the reference labels independently, before any method is run on the held-out pairs. Draw held-out data from a later window and from projects outside the ledger. Report fixes made after the freeze as a separate run. The work proceeds in this order: the conformance suite (done; Section 6); the primary applicability comparison against (iv), with (v) as the secondary test S1; the agent experiment; cost measurement.

Independence. Independence is the hardest requirement of this design. The annotators who label pairs and the curators who build the reference set should not be the people who develop the detector. The roles open to outside participants are listed under Availability and Collaboration.

11 Q3: Retrospective Study on Historical Changes

This section reports an exploratory, retrospective study (R-C0) that applies OpenPulse to real upstream changes from 2016–2025 and to real dependency references read at commits from before each change. It is *not* the prospective evaluation of Section 10, it is not pooled with it, and it is

Policy	False clears (events)	Silent miss	False impact	Resolved	Abstained	Unsupp. decisive
A naive	35 (10)	0.0%	3.2%	77.6%	0.0%	100%
B conservative	0 (0)	0.0%	3.2%	41.2%	58.1%	0%
V strong conservative	0 (0)	0.0%	1.6%	47.3%	52.1%	0%
C OpenPulse at the freeze	0 (0)	11.7%	1.6%	50.3%	38.9%	0%
C after fixes (post hoc)	0 (0)	0.0%	1.6%	58.2%	38.3%	0%

Table 23: Retrospective study: 167 pairs over 27 events. *Silent miss*: in-scope pairs with no association at all, over in-scope pairs. *False impact*: out-of-scope pairs decided affected, over out-of-scope pairs. *Resolved*: correct decisive decisions over determinable pairs. *Abstained*: unresolved over all pairs. *Unsupp. decisive*: decisive outcomes on the two undeterminable pairs. The last row is a separate run after two fixes and is not the study's result.

not evidence of effectiveness in deployment. Its design, every amendment, the frozen case file and the results are in `evaluation/retrospective/` of the repository; this section summarises `REPORT.md` there.

Design. Twenty-seven events were screened into six classes under quotas fixed before any data were collected: relocation (3), deprecation and archival (6), distribution (6), licence change (4), end of life (3) and compromised package versions (5). Each event is a fixture built only from archived primary sources (Wayback snapshots or immutable records) and replayed at its snapshot date, so it is evaluated as it was knowable then. For each event we collected real references that should be in scope and nearby references that should not, each a file in a public repository at a stated commit inside the event's window: 102 in-scope references and 65 neighbours, none synthetic, giving 167 pairs. One reference fell outside its window and was excluded; no event was dropped. Labels follow one written rule per event, taken from that event's primary source: 103 in scope, 62 out of scope and 2 undeterminable. The implementation was frozen at tag `retro-freeze-2026-10-30` (`3639a0f`) and the case file before any policy ran (SHA-256 recorded and checked in CI). The run used the pinned ecosystem comparators (`node-semver` 7.8.5, `golang.org/x/mod` v0.42.0, `maven-artifact` 3.9.16) and a 90-day freshness bound. A second run reproduced the results file byte for byte.

Labels. All labels are the developer's. Two rounds of outside labelling on a seeded 25% sample (42 pairs) turned out to be AI-assisted: the second round's method note states that a language model prepared the labels. We therefore report them only as a consistency check (agreement with the developer's labels $\kappa = 0.722$ and 0.825), never as inter-rater reliability, and no pair is marked as independently labelled. The pre-specified analysis, whose population is independent labels only, is consequently empty, and every test below is illustrative.

Policies. A (naive) reports affected if any current event lists the dependency, and otherwise not affected. B (conservative) abstains instead of clearing; it corresponds to the minimal reference (iv) and is the primary comparator of the registration draft (Section 10). V is the strong conservative baseline (v), compared with C by the equivalence test S1. C is OpenPulse. The registration draft's thresholds are applied below for illustration only.

Decisions. Table 23 gives the result at the freeze. C issued no false clear on any of the 27 events, where A falsely cleared 35 pairs in 10 events; the event-level difference is significant (Holm-adjusted $p = 0.0004$). With 27 events the one-sided 95% upper bound on C's per-event

false-clear share is 0.105, above the pre-specified margin, and 27 is below the 60 events the pre-specified safety test needs, so non-inferiority is not decidable here. The gain in resolution over B (0.091, 95% CI $[-0.054, 0.245]$) misses the pre-specified 0.10, and equivalence with V within ± 0.05 is not shown (0.030, 90% CI $[0.000, 0.075]$). C's advantage is concentrated in end of life (88.2% resolved against V's 58.8%). C decided neither undeterminable pair, and its one false impact, an Early Access `openjdk` tag that the announcement excludes, comes from a representation gap recorded before the run.

A defect the study exposed. Guardrail G1 (silent miss ≤ 0.02) fails: 12 of 103 in-scope pairs (11.7%) received no association. Every one is a Go module reference, such as `github.com/docker/docker`, to a repository the event names as a source-repository artefact. The engine never compared a Go module path with a repository artefact, although a Go module path is a repository locator. V associated all twelve. A silent miss is not a false clear, because nothing is reported either way, but it withholds the warning the event exists to give.

Where C abstained. Of C's 82 non-correct decisions on determinable pairs, 17 come from that defect, 44 from packages with no catalogued identity (all 31 compromised-package pairs among them, which the identity rule of Section 5.5 leaves unresolved by design), 20 from representation gaps (19 unresolved and the one false impact), and 1 is a correct non-association with a different project. The representation gaps are scope forms the specification cannot yet state exactly: registry-wide scopes (only as envelopes), open-ended version boundaries ("every release from 1.6.0"), a date boundary across release branches (MongoDB's SSPL change), and exclusions (Early Access tags). The fixtures state each as a safe over-approximation, which costs resolution and never safety.

Discovery. For the 15 relocation, archival and distribution events, OpenPulse's live collectors detected none as shipped, because their projects were not catalogued and collection was keyed by catalogue entry. Given each repository or image directly, they detected 5, all archivals, each with a project-wide scope broader than the announced one. They missed every relocation: GitHub answers a transferred repository with a 301 redirect, which the collector recorded as an error, so the ownership comparison of Section 9 never ran. Distribution changes stated only in a notice, a registry other than Docker Hub, and tag deletions visible only across two observations were outside what the collectors observe.

After fixes. Both defects were fixed in their own change (a Go module now reaches the repository it lives in, positively only, and the collector follows one redirect hop on the API origin). The frozen case file was then run again on the freeze tag plus those two fixes and nothing else. Thirteen decisions changed, all to correct; G1 passes; the gain over B becomes 0.170 (one-sided lower bound 0.031); C leads V by 0.109; there is still no false clear; and discovery detects all three relocations, naming exactly the announced artefact. These figures are post hoc: the fixes were written after seeing the errors on the same events, so they show what caused the errors, not unbiased performance.

Where C and V differ. After the fixes, C and V decide 23 of the 167 pairs differently, and only through the two mechanisms named for (v) in Section 10. Eighteen are Go module references to a relocated or deprecated repository: V leaves all of them unresolved, while C reports the 13 in scope as affected and gives the 5 out-of-scope neighbours (the new location or a different project) no association. Five are image tags with a variant suffix in two end-of-life events (`node:12-alpine`, `node:16-alpine`, `python:3.7-slim`): V cannot compare them, and C decides all five correctly, three affected and two cleared. Every difference favours C under the developer's labels, and none involves completeness, identity or admission, which the two share.

The study therefore gives no evidence that OpenPulse's value lies in its engine rather than in the shared model; at the freeze, before the Go fix, the two differed on 22 pairs and C's lead was 0.030.

A false clear outside the study's measurement. The study's C applies events only; it passes no advisory data, so OpenPulse's second decision path, which correlates OSV and NVD advisories by package and version, never ran. Closing the compromised-package gap by querying OSV per package then exposed a defect in that path that the audit of Section 6 had not found. Advisories were merged by CVE identifier, and an advisory without one was dropped. Most advisories for malicious or sabotaged packages have none, so `node-ipc` 9.2.2, the version that shipped the protestware dependency, was reported NotAffected against an unrelated CVE. Two false-impact defects in the same reader were found alongside it: an advisory's range for one package was applied to another package listed in it, and an advisory listing only affected versions left every other version undetermined. All three are fixed, with regression tests that fail on the old code. The study's safety result therefore holds for event-scope decisions only; the advisory path needs its own evaluation, with advisories as they stood at each event's date.

12 Limitations

Evidence of value. No independent effectiveness result exists. The ledger was built together with the detector, it has no denominator, and its labels were not independently adjudicated. Confidence levels are uncalibrated policy categories. The retrospective study of Section 11 has a denominator but not independence: every label is the developer's, its events were researched under quotas rather than sampled from a population, its dependents were found among repositories that still exist, and the fixtures were written by someone who knows the scope language. Its 27 events cannot decide the pre-specified safety test, which needs 60. It measured the event-scope path only; the advisory path, which held a false-clear defect (Section 11), was not exercised.

Soundness. The lemma is conditional, and several premises are not established (Table 11). The report shows that the aggregation is sound under its premises, not that the deployed system meets them often enough to justify its negatives. Completeness is declared, never verified, though each declaration now records its basis. The freshness bound is off unless the caller sets it, and the implementation's regression tests are chosen cases. The conformance matrix covers chosen dimensions, and its oracle was written by the same team. Interim rules withhold negatives where semantics are missing (OCI levels), at the cost of more undetermined verdicts. All fifteen enumerated defects have regression fixes. Known gaps in the negative-admission conditions remain: freshness is opt-in outside the agent interface; conflicts between separate events are not detected; package names are not canonicalised at `8a9c3a8`, and are made undetermined instead (at `a1ecc58` they are compared per ecosystem, and qualified values decide by ecosystem and name; Table 1); and declarations are recorded but not validated. Six of the twelve were found after the initial audit: four while answering external questions, and two by the conformance matrix. That increases concern that additional defects remain, and the retrospective study confirmed it: it exposed a false clear in the advisory path, which the audit had not examined with advisories lacking a CVE identifier, and a silent-miss defect in the event path (Section 11). Closing D10 makes every unmapped dependency Related. Its coverage cost is measured only per pair, in the retrospective study, where uncatalogued packages caused 44 of C's 82 non-correct decisions. It is unmeasured per inventory and per monitoring cycle, which is what determines review workload: an inventory with a long tail of private or repackaged components could turn most of its references into low-information associations that are each defensible and together burdensome. A trivial always-abstaining policy has no false clears, so the question is whether OpenPulse makes materially more justified decisions than (v) at an acceptable review burden.

Package-name canonicalisation is incomplete. Element roles follow from form and are not labelled in the contract, and the tag-predicate language has two predicates only. Version comparison covers SemVer, PEP 440 and plain numbers. Other ecosystems' orderings are undetermined rather than modelled. PEP 440, npm, Go and Maven ordering are tested differentially against reference implementations; the rest are not. The audit covered the paths examined. Without a freshness bound, the assurance a negative provides is limited to evidence whose currency the consumer has checked through `last_verified`.

Scope. Discovery covers three classes on two platforms, uses English-language sources only, and cannot see private registries. Inventory-driven discovery (`a1ecc58`) maps only Go module paths, `github.com` references and Docker Hub images to watch targets; vanity import paths, other registries, deprecations stated only in a registry flag or a notice, and tag deletions seen only across observations remain outside it. No advance warning has been demonstrated for an unrecorded class.

Integrity and consumption. Local evidence and contracts are unauthenticated. First detections can be backdated, and a privileged local operator is outside the integrity model. The agent interface is an unvalidated integration boundary, not a security mitigation. It is untested against injection, and the structural-separation mitigation of Section 7 is designed but not implemented. Continuous monitoring at scale lacks backoff and request scheduling. Matching has no index, so cost grows with scope size. The observation store keeps full snapshots, and each sweep reads the whole history. Costs were measured on one machine, not at population scale.

13 Conclusion

Dependency decisions increasingly need evidence about upstream changes that vulnerability and lifecycle records do not carry, and some of those decisions are now proposed by AI agents. OpenPulse collects that evidence, qualifies it, and applies it to inventories under a specification in which a negative requires a complete scope. A short lemma makes that aggregation sound given premises the implementation must establish. Our audit of the implementation found fifteen defects, twelve of them ways in which a false clear could arise or be displayed, some of which the regression suite had treated as correct. All have regression fixes, and the negative-admission conditions are enforced, with stated gaps. An executable oracle written from the specification now agrees with the implementation on 100,000 random cases and on every matrix case apart from one documented, non-blocking label difference (72 of 6,195 cases). It catches every re-injected historical-defect and rule-disabling mutant, and it guards every change as a build gate. Building it also corrected the specification in four places. That is why conformance and differential testing must come before claims of effectiveness. A first retrospective study on 27 real changes found no false clear by the event-scope engine where a naive reader issued 35, and it also found what conformance alone could not: a reference form the engine never matched, a discovery path that saw nothing it had not been told about, and a false clear in a decision path the audit had not exercised. Each is fixed and reported. Whether the evidence changes dependency decisions, for developers or for agents, is the question the evaluation in Section 10 is designed to answer.

Availability and Collaboration

OpenPulse is open source under the Apache License 2.0 at `https://github.com/IkarusAILabs/OpenPulse`. It includes its schemas, recorded fixtures and replay tests. It is developed by Ikarus AI Labs, a non-profit research initiative based in Cambridge, UK, as part of a programme on evidence-based assurance for AI-mediated software change. We welcome

collaboration, in particular from researchers and practitioners independent of the detector's development. Useful roles are independent annotation and reference-set curation, contributing real inventories that contain distributions and embedded components, running the agent experiment, and integrating the evidence contract into dependency-decision workflows and assurance tooling. See `https://ikaruslabs.net/`.

References

[1] Afsah Anwar, Ahmed Abusnaina, Songqing Chen, Frank Li, and David Mohaisen. Cleaning the NVD: Comprehensive quality assessment, improvements, and analyses. *IEEE Transactions on Dependable and Secure Computing*, 19(6):4255–4269, 2022.

[2] Guilherme Avelino, Eleni Constantinou, Marco Tulio Valente, and Alexander Serebrenik. On the abandonment and survival of open source projects: An empirical investigation. In *2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–12. IEEE, 2019.

[3] Bitnami. Upcoming changes to the Bitnami catalog (effective August 28th, 2025). GitHub issue bitnami/containers#83267, July 2025. Opened July 16, 2025. Accessed 2026-10-28.

[4] Bodin Chinthanet, Raula Gaikovina Kula, Shane McIntosh, Takashi Ishio, Akinori Ihara, and Kenichi Matsumoto. Lags in the release, adoption, and propagation of npm vulnerability fixes. *Empirical Software Engineering*, 26(3), 2021.

[5] Jürgen Cito, Gerald Schermann, John Erik Wittern, Philipp Leitner, Sali Zumberi, and Harald C. Gall. An empirical analysis of the Docker container ecosystem on GitHub. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*, pages 323–333. IEEE, 2017.

[6] E. F. Codd. Extending the database relational model to capture more meaning. *ACM Transactions on Database Systems*, 4(4):397–434, 1979.

[7] Filipe R. Cogo, Gustavo A. Oliva, and Ahmed E. Hassan. Deprecation of packages and releases in software ecosystems: A case study on npm. *IEEE Transactions on Software Engineering*, 48(7):2208–2223, 2022.

[8] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252. ACM, 1977.

[9] Scott A. Crosby and Dan S. Wallach. Efficient data structures for tamper-evident logging. In *18th USENIX Security Symposium*. USENIX Association, 2009.

[10] Cybersecurity and Infrastructure Security Agency. Known exploited vulnerabilities catalog. CISA. Accessed 2026-10-28.

[11] Cybersecurity and Infrastructure Security Agency. Minimum requirements for vulnerability exploitability eXchange (VEX). Technical report, CISA, April 2023. Published April 21, 2023.

[12] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents, 2024.

[13] Alexandre Decan, Tom Mens, and Philippe Grosjean. An empirical comparison of dependency network evolution in seven software packaging ecosystems. *Empirical Software Engineering*, 24(1):381–416, 2019.

[14] Ecma International. ECMA-424: CycloneDX bill of materials specification. Standard, Ecma International, December 2025.

[15] endoflife.date contributors. endoflife.date. Website and API. Accessed 2026-10-28.

[16] Oren Etzioni, Keith Golden, and Daniel S. Weld. Sound and efficient closed-world reasoning for planning. *Artificial Intelligence*, 89(1–2):113–148, 1997.

[17] GitHub. GitHub advisory database. GitHub repository github/advisory-database. Advisories stored in OSV format. Accessed 2026-10-28.

[18] GitHub. Transferring a repository. GitHub Docs. Accessed 2026-10-28.

[19] Ilay Goldman and Yakir Kadkoda. GitHub dataset research reveals millions potentially vulnerable to RepoJacking. Aqua Security (Aqua Nautilus) blog, June 2023. Published June 21, 2023. Accessed 2026-10-28.

[20] Google. Open source insights (deps.dev). Service and API documentation. Accessed 2026-10-28.

[21] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90. ACM, 2023.

[22] Yacong Gu, Lingyun Ying, Yingyuan Pu, Xiao Hu, Huajun Chai, Ruimin Wang, Xing Gao, and Haixin Duan. Investigating package related security threats in software registries. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1578–1595. IEEE, 2023.

[23] Tomasz Imieliński and Witold Lipski. Incomplete information in relational databases. *Journal of the ACM*, 31(4):761–791, 1984.

[24] Nasif Imtiaz, Seaver Thorn, and Laurie Williams. A comparative study of vulnerability reporting by software composition analysis tools. In *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–11. ACM, 2021.

[25] Elgun Jabrayilzade, Mikhail Evtikhiev, Eray Tüzün, and Vladimir Kovalenko. Bus factor in practice. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 97–106. ACM, 2022.

[26] Stephen Cole Kleene. *Introduction to Metamathematics*. North-Holland, 1952.

[27] Arjun Krishna, Erick Galinkin, Leon Derczynski, and Jeffrey Martin. Importing phantoms: Measuring LLM package hallucination vulnerabilities, 2025.

[28] Raula Gaikovina Kula, Daniel M. German, Ali Ouni, Takashi Ishio, and Katsuro Inoue. Do developers update their library dependencies? *Empirical Software Engineering*, 23(1):384–417, 2018.

[29] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. SoK: Taxonomy of attacks on open-source software supply chains. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1509–1526. IEEE, 2023.

[30] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, RFC Editor, June 2013. Experimental; obsoleted by RFC 9162.

[31] Ben Laurie, Eran Messeri, and Rob Stradling. Certificate transparency version 2.0. RFC 9162, RFC Editor, December 2021. Experimental.

[32] Alon Y. Levy. Obtaining complete answers from incomplete databases. In *Proceedings of the 22nd International Conference on Very Large Data Bases (VLDB)*, pages 402–412, 1996.

[33] Frank Li and Vern Paxson. A large-scale empirical study of security patches. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2201–2215. ACM, 2017.

[34] Changyuan Lin, Sarah Nadi, and Hamzeh Khazaei. A large-scale data set and an empirical study of Docker images hosted on Docker Hub. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 371–381. IEEE, 2020.

[35] Courtney Miller, Christian Kästner, and Bogdan Vasilescu. "We feel like we're winging it:" a study on navigating open-source dependency abandonment. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1281–1293. ACM, 2023.

[36] Amihai Motro. Integrity = validity + completeness. *ACM Transactions on Database Systems*, 14(4):480–502, 1989.

[37] National Institute of Standards and Technology. National vulnerability database (NVD): Program updates. NIST. Status notices on the CVE enrichment backlog since early 2024, incl. updates of March 19, 2025 and April 15, 2026. Accessed 2026-10-28.

[38] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2353–2367. ACM, 2022.

[39] OASIS. Common security advisory framework version 2.0. Oasis standard, OASIS Open, November 2022.

[40] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. Backstabber's knife collection: A review of open source software supply chain attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2020)*, Lecture Notes in Computer Science, pages 23–43. Springer, 2020.

[41] OpenSSF. Open source vulnerability (OSV) format. Schema specification, version 1.9.2. Original authors Oliver Chang and Russ Cox. Accessed 2026-10-28.

[42] OpenVEX Project. OpenVEX specification. GitHub repository openvex/spec. Draft; JSON-LD context v0.2.0. Accessed 2026-10-28.

[43] Ivan Pashchenko, Henrik Plate, Serena Elisa Ponta, Antonino Sabetta, and Fabio Massacci. Vulnerable open source dependencies: Counting those that matter. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–10. ACM, 2018.

[44] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 754–768. IEEE, 2022.

[45] Henrik Plate, Serena Elisa Ponta, and Antonino Sabetta. Impact assessment for vulnerabilities in open-source software libraries. In *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 411–420, 2015.

[46] Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. Beyond metadata: Code-centric and usage-based analysis of known vulnerabilities in open-source software. In *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 449–460. IEEE, 2018.

[47] Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. Detection, assessment and mitigation of vulnerabilities in open source dependencies. *Empirical Software Engineering*, 25(5):3175–3215, 2020.

[48] Simon Razniewski and Werner Nutt. Completeness of queries over incomplete databases. *Proceedings of the VLDB Endowment*, 4(11):749–760, 2011.

[49] Raymond Reiter. On closed world data bases. In *Logic and Data Bases*, pages 55–76. Springer, 1978.

[50] Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2):159–176, 1999.

[51] Rui Shu, Xiaohui Gu, and William Enck. A study of security vulnerabilities on Docker Hub. In *Proceedings of the Seventh ACM Conference on Data and Application Security and Privacy (CODASPY)*, pages 269–280. ACM, 2017.

[52] SLSA Community. SLSA specification, version 1.2. Supply-chain Levels for Software Artifacts. Status: Approved. Accessed 2026-10-28.

[53] Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 3687–3706. USENIX Association, 2025.

[54] Trevor Stalnaker, Nathan Wintersgill, Oscar Chaparro, Massimiliano Di Penta, Daniel M. German, and Denys Poshyvanyk. BOMs away! inside the minds of stakeholders: A comprehensive study of bills of materials for software systems. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*, 2024.

[55] The Linux Foundation SPDX Project. The software package data exchange (SPDX) specification. Current version 3.0; the SPDX specification is an international open standard (ISO/IEC 5962:2021). Accessed 2026-10-28.

[56] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1393–1410. USENIX Association, 2019.

[57] Marat Valiev, Bogdan Vasilescu, and James Herbsleb. Ecosystem-level determinants of sustained activity in open-source projects: A case study of the PyPI ecosystem. In *Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 644–655. ACM, 2018.

[58] Boming Xia, Tingting Bi, Zhenchang Xing, Qinghua Lu, and Liming Zhu. An empirical study on software bill of materials: Where we stand and the road ahead. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, pages 2630–2642, 2023.

[59] Nusrat Zahan, Parth Kanakiya, Brian Hambleton, Shohanuzzaman Shohan, and Laurie Williams. OpenSSF scorecard: On the path toward ecosystem-wide automated security metrics. *IEEE Security & Privacy*, 21(6):76–88, 2023.

[60] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. What are weak links in the npm supply chain? In *2022 IEEE/ACM 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 331–340. IEEE, 2022.

[61] Ahmed Zerouali, Tom Mens, Gregorio Robles, and Jesus M. Gonzalez-Barahona. On the relation between outdated Docker containers, severity vulnerabilities, and bugs. In *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 491–501. IEEE, 2019.

[62] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. Small world with high risks: A study of security threats in the npm ecosystem. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 995–1010. USENIX Association, 2019.

A Worked Example: etcd Relocation

Serialised event. From the fixture `data/fixtures/etcd-move/` (repository metadata probed on 2026-10-06), the analyst emits a relocation event. Its serialised type is `OWNERSHIP_CHANGE`, a schema name that predates this report. The event asserts only that the recorded path now resolves elsewhere; as Section 9 states, path divergence does not establish a change of controlling principals, and the name should not be read as one. A relocation-specific type is a schema change we have not yet made.

```
affected_artifacts: [{kind: source-repository, ref: github.com/coreos/etcd}]
scope: {kind: project, versions: []} confidence: EMERGING impact: REVIEW
```

In the typed model the scope is the project element `etcd`, and the repository `github.com/coreos/etcd`, carried in `affected_artifacts`, is a *named example*: it illustrates the change rather than bounding it (Section 5). χ is partial: relocation events do not declare completeness, and none of the verdicts below is a negative.

Three verdicts by Algorithm 1.

- `github.com/coreos/etcd`: the named repository gives M, so the verdict is AffectsArtifact (confidence Emerging).
- Package `etcd` with no version: the named repository gives 0 (a named example does not relate to another reference kind) and the project element gives P, so the verdict is AffectsProject.
- `github.com/etcd-io/etcd`: the named repository gives 0, since a named example says nothing about other repositories. The new canonical path does *not* resolve to project `etcd`: it is an unmapped reference that resolves to itself. Its identity is unresolved, so the project element gives ?, and the verdict is Related.

Specification and implementation agree on all three: the oracle of Section 6 returns the same verdicts for this event, and `test_golden_etcd_namespace_move` pins the implementation's. Earlier drafts modelled the repository as an exact scope element, under which the specification would give Related for the package and the case appeared as a discrepancy. That reading predates correction S4, which fixed the role of named examples; it is not a remaining difference, and the only one at either snapshot is the label difference of Section 6.

Unjustified negative, unresolved result, justified exclusion. Table 24 traces the three outcomes the report distinguishes through one event, Bitnami's catalogue-wide withdrawal (`data/fixtures/bitnami/event.json`), plus one unmapped reference. The canonical identity of every reference is computed before any test (D9). For each verdict OpenPulse retains an evidence contract. It carries the event and its cited sources, the identity with its trust status, the scope, the matching element and match method, the assessment, the unknowns, the temporal roles, and a content hash. The first detection is also recorded in the ledger.

Reference and scope	Element tests and completeness	Verdict	Outcome
`bitnami/nginx:1.25` against the five-image list read as complete	Every listed image gives `N`; χ assumed, no basis	NotAffected (at `a461bc9`)	Unjustified negative (D4)
`docker.io/zzz/kvstore:3.5` against an etcd 3.5 end-of-life scope	Identity unresolved: the reference resolves only to itself, so it gives `?`	Related	Unresolved, correctly
`docker.io/redis:7.2` against the namespace elements under `docker.io/bitnami`, `docker.io/bitnamilegacy` and the chart registry	Every namespace element gives `N` by location; χ declared, announced basis	NotAffected	Justified exclusion, under a policy admitting announced scopes

Table 24: Three outcomes across two events. At `8a9c3a8` the same Bitnami scope makes `bitnami/nginx:1.25` AffectsArtifact, through the exact element `docker.io/bitnami/*:<version>`, and `bitnami/redis:latest` Related, through the envelope `docker.io/bitnami/*`.

B Defect Details

D4 in curated data. Implicit completeness also affected the curated evidence. The Bitnami event's scope listed five *example* images of a catalogue-wide change, and because the list was read as complete, an unlisted image such as `docker.io/bitnami/nginx` was cleared. Marking the list partial would have made every container image Related to that event. The correct repair was to state the scope the announcement actually describes, using namespace elements declared complete. At `8a9c3a8` they are `docker.io/bitnami/*:<version>` (exact) and `docker.io/bitnami/*` (envelope), `docker.io/bitnamilegacy/*:*`, and the chart registry `bitnamicharts/*:*`. With that scope, `bitnami/nginx` is affected, `bitnami/redis:latest` is Related, and `docker.io/redis` is cleared.

D7: tags. Artefact elements match on the repository triple and ignore tags. For the Bitnami event, which names `bitnami/redis:7.2` among its affected artefacts, the engine therefore reports `bitnami/redis:5.0.4` as "matching" `7.2`. That outcome is correct only because the event is repository-wide. It also reports `bitnami/redis:latest` as affected, although `latest` is what remained in the mainline. The result was a false impact with an overstated reason, not a false clear. The repair in `bfb1ee9` has three parts. Element roles make the plain namespace an envelope. The tag predicate `<version>` restates the exact part of the scope. And a named literal tag now establishes that tag only, so `bitnami/redis:latest` is Related and `bitnami/redis:5.0.4` is affected through the versioned-tag element.

D8: numeric pre-release identifiers. The generic scheme splits a version into digit and letter runs. A numeric component after `-` or `+` is therefore folded into the numeric core: `1.0.0-1`

becomes $(1, 0, 0, 1)$ and sorts *after* `1.0.0`. Under SemVer, `1.0.0-1` is a pre-release and sorts *before* `1.0.0`. Against an OSV `SEMVER` range with `introduced: 0, fixed: 1.0.0`, the versions `1.0.0-0` and `1.0.0-1` are reported outside the range although they are inside it. That is a false clear. The same folding makes `1.0.0+5` (build metadata, which SemVer ignores for ordering) sort above `1.0.0`, and it treats Debian revisions such as `2.4.1-1` as an extra numeric component. The scheme also ignored the number after a marker, so `1.0.0-rc1` equalled `rc2`, and a fix in `rc2` cleared `rc1`. The repair in `9d53ce2` is a closed grammar: a version is comparable only as a dotted numeric core followed by at most one recognised marker and an optional number. Anything else, including `1.0.0-1`, `2.4.1-1` and `1.0.0+5`, is undetermined, and `rc1` sorts below `rc2`. That conservative repair was an intermediate state for SemVer ranges: since D11 (below), an OSV `SEMVER` range is compared by SemVer precedence, under which `1.0.0-1` is inside the range, as it should be. The closed grammar still governs the generic scheme. D8 was not caught by the range tests, which we wrote from our own reading of the specification. It is the kind of error that the version differential (Section 6) catches: its mutant is caught there.

D9: registry host aliases. Reference canonicalisation lowercases a reference, strips a URL scheme, a digest and the tag of the last path segment, defaults a missing registry to `docker.io`, and gives a single-segment name the namespace `library`. It did not fold the aliases under which Docker Hub also serves images, `index.docker.io` and `registry-1.docker.io`, into `docker.io`. Against the Bitnami event's namespace elements, declared complete, `docker.io/bitnami/nginx` was affected while `index.docker.io/bitnami/nginx`, the same image, was cleared. That was a false clear. Exact comparison is only as meaningful as the canonicalisation in front of it. The specified repair is a canonical form per element kind: registry host aliases, namespace and `library`, package-name normalisation per ecosystem, curated aliases, and digest algorithm. The executable oracle and the conformance suite need the same form. The fix in `7c0b9d9` folds both Docker Hub hosts into `docker.io` in the one normaliser that matching and identity share. Package-name normalisation per ecosystem is not part of the canonical form at `8a9c3a8`; it is at `a1ecc58` (F1–F3, Table 1).

D10: unmapped identity. An unmapped reference resolves only to itself, and the identity layer marked that self-resolution VERIFIED, as if it established a different project. Two paths followed. Under a version-scoped event, such as an end-of-life notice for etcd 3.5, an unmapped mirror `docker.io/zzz/kvstore:3.5` was excluded on identity grounds and cleared per pair. Under a project-wide event it was UNKNOWN, unless its name resembled the project, and dropped from the summary, so another event's clearance headlined. The fix in `7c0b9d9` makes every self-resolved reference `?`, whatever its name. Three regression tests had used unmapped references to stand for "another project". They now use references curated as other projects. One test asserted the name rule itself, and it is reversed.

D11: pre-release ordering under SemVer. The generic grammar ranks pre-release markers by a fixed table: `dev` below `a`/`alpha`, below `b`/`beta`, below `c`/`rc`/`pre`/`preview`. It treats `a` and `alpha`, and `c` and `rc`, as equal, as PEP 440 does. SemVer orders alphanumeric pre-release identifiers lexically instead. So `1.0.0-dev` sorts *above* `1.0.0-beta`, `a` below `alpha`, and `c` below `rc`. The generic comparator is applied to OSV `SEMVER` ranges as well, and three false clears follow. For example, `1.0.0-dev` against `introduced: 1.0.0-beta, fixed: 1.0.1` is reported outside the range although SemVer places it inside. Accepting a form is therefore not enough. The form must be supported under the range's own comparison scheme, and its ordering must agree with that scheme. The repair in `dabafef` dispatches on the range type and ecosystem, and returns `?` wherever no implemented scheme reproduces the ecosystem's ordering (Section 5).

D12–D14: found by the conformance matrix. D12: a reference whose first segment is a registry host was treated as a registry reference only with three or more segments. So `docker.io/redis`, Docker's own spelling of `docker.io/library/redis`, was read as namespace `docker.io`, and complete scopes naming the official image cleared it. D13: an unmapped package was dropped as unrelated to other projects' events, the package-side twin of D10. It is now unresolved against project-wide, release-line and package scopes, and has no association with container-image and registry scopes. Two packages that tests had used to mean "a dependency nothing applies to" are now curated as their own projects. D14: a repository element that names no tag gave RELATED for a digest-pinned reference to the repository, though it covers every pin. All three are fixed in `784fed8`.

C Reproducibility

- Repository: `https://github.com/IkarusAILabs/OpenPulse`. Pinned commit: `8a9c3a829ad0da7a8a1e935f677af0eb8810ed0c` (`main`, 29 October 2026). D1–D3 and D5 were fixed in `4b92f7d`, D6 in `b5cbbde`, D4 and the interim rules in `f0c3b48`, D8 in `9d53ce2`, D9 and D10 in `7c0b9d9`, D11 and the admission conditions in `dabafef`, D7 and the element roles in `bfb1ee9`, and D12–D14 in `784fed8`, D15 in `d4f7d1f`. Audited commit: `a461bc99a6edebbdb124e4b9a3c477282aae45a6`. The latest tagged release, v0.7.1, predates the fixes. Licence: Apache-2.0.
- Current snapshot: `a1ecc58` (`main`, 30 October 2026), with the changes of Table 1; evidence contract 1.2.0, intelligence schema 0.7.0, 1,080 tests collected, all passing in CI. The conformance matrix, randomised cases, re-injection, version differential, aggregation check and cost benchmark were re-run there with the commands below; the audit harness and the 953-test count refer to the pinned commit.
- Retrospective study (Section 11): freeze tag `retro-freeze-2026-10-30` (`3639a0f`); case file, its SHA-256, results and the CI workflow that reproduces them in `evaluation/retrospective/`.
- Environment: Python ≥3.10 with the hash-locked `requirements.lock`; all tests run offline. Intelligence schema 0.7.0, evidence contract v1.1 at `8a9c3a8` and 1.2.0 at `a1ecc58`.
- Full suite at the pinned commit: 953 tests pass. Test count is not evidence of semantic completeness.
- Case ledger: `pytest tests/test_golden.py`. The recorded evidence is under `data/fixtures/`.
- Audit harness: the regression tests do not exist at the audited commit, so they are transplanted. The script `papers/tr-2026-01/reproduce_audit.sh` does this from a clean clone. It checks out `a461bc9` and `8a9c3a8` as separate worktrees, copies `tests/test_false_clear.py` from `8a9c3a8` into the audited one, and runs it in both. Expected result: at `a461bc9`, 52 tests fail (the defect, interim-rule, admission and role tests) and 12 pass; the passing ones pin behaviour the fixes must preserve. The withdrawn-advisory test passes at `a461bc9` only because D2 hides it. At `8a9c3a8` all 64 pass. Table 25 gives the breakdown by defect.
- Conformance: `python conformance/matrix.py` and `python conformance/versions_differential.py`. Results, including every disagreement, are in `conformance/results/` and are analysed in `conformance/REPORT.md`. Randomised testing: `python conformance/randomized.py -cases 100000 -seed 7`. Defect re-injection: `python conformance/mutation.py`. The build gate is `tests/test_conformance_gate.py`.
- Cost: `python benchmarks/cost.py` (offline), with `-live` for acquisition. The results and the machine are recorded in `benchmarks/REPORT.md`.
- Cross-event aggregation: `python papers/tr-2026-01/check_aggregation.py` checks the summary function against Table 10. Expected: 19,607 sequences, 13,699 set orderings, 0

mismatches.

- D8: `osv_applicable("1.0.0-1", {"ranges": [{"type": "SEMVER", "events": [{"introduced": "0"}, {"fixed": "1.0.0"}]}]})` in `core/versions.py` returns `False` (cleared) at `f0c3b48`, `None` (undetermined) at the D8 repair `9d53ce2`, and `True` (inside) from the D11 repair `dabafef` on, including `8a9c3a8` and `a1ecc58`.
- D9: `check_dependency({"kind": "image", "ref": "index.docker.io/bitnami/nginx:1.25"}, [event])`, with the event from `data/fixtures/bitnami/event.json`, returns NotAffected at `9d53ce2` and AffectsArtifact at `8a9c3a8`, like its `docker.io` spelling.
- Snapshot: a commit identifier is not a release. The submission snapshot will be a tagged, archived release containing the specification and schema versions, the regression tests with their expected outcomes, the raw recorded evidence for the ledger (subject to source terms), the conformance oracle, the metric scripts, and a manifest of the exact implementation and data versions used in each experiment.
- Contract hash: `verify_v1_content_hash` in `core/evidence_contract.py` recomputes it with the hash and timestamp fields pinned to constants (`docs/EVIDENCE_CONTRACT.md`).

Defect	Tests in `test_false_clear.py`	Fail at `a461bc9`	Fixed	Residual limitation
D1	`test_d1_*`	4 / 4	`4b92f7d`	OCI levels unresolved (interim ?)
D2	`test_d2_*`	4 / 9	`4b92f7d`	`GIT` and unknown range types undetermined
D3	`test_d3_*`, `test_package_scoped_*`	2 / 4	`4b92f7d`	–
D4	`test_d4_*`	3 / 4	`f0c3b48`	Declarations unverified
D5	`test_d5_*`	2 / 2	`4b92f7d`	A wrong curated mapping is not detected
D6	`test_d6_*`	3 / 3	`b5cbbde`	Epochs undetermined, not compared
Interim	`test_interim_*`, `test_namespace_*`	7 / 9	`f0c3b48`	OCI resolution open
D7, roles	`test_d7_*`, `test_envelope_*`, `test_version_tag_*`, `test_bitnami_*`	5 / 5	`bfb1ee9`	Roles by form, not labelled in the contract
D8	`test_d8_*`	4 / 4	`9d53ce2`	One generic scheme; ecosystem forms undetermined
D9	`test_d9_*`	2 / 2	`7c0b9d9`	Package-name forms not canonical
D10	`test_d10_*`	4 / 4	`7c0b9d9`	Coverage cost unmeasured; curation needed
D11	`test_d11_*`	3 / 3	`dabafef`	Other ecosystems' orderings undetermined
N3–N5	`test_n3_*`, `test_n4_*`, `test_n5_*`, `test_package_name_*`	6 / 7	`dabafef`	Freshness opt-in; cross-event conflicts
D12–D14	`test_d12_*`, `test_d13_*`, `test_d14_*`	3 / 4	`784fed8`	Uncatalogued packages now Related
Total	64 tests	52 / 64		

Table 25: Audit reproducibility by defect. Passing tests at `a461bc9` pin behaviour that the fixes must preserve.